

面向不确定性的绿色物流配送鲁棒性优化： 从绿区限行到实时动态重调度

摘 要

本文聚焦于某省会城市物流服务商的日常配送场景、围绕异质车队静态调度、绿色配送区限行政策影响评估和突发事件动态响应三个递进问题，构建了一套随机时变车辆路径规划（SVRP）建模与求解体系。

针对问题一，本文模型以配速而非速度作为随机原型，规避正态倒数分布的矩发散问题，使弧段旅行时间成为配速正态变量的线性组合。在此基础上，利用截断正态矩推导软时间窗早到与迟到惩罚的闭式期望，构造包含启动成本、能耗及碳税成本、时间窗惩罚的总成本极小化目标函数，并施加五型车队容量、流量平衡与 MTZ 消环等约束。由于 36 位客户的聚合需求超出单车载重上限，模型允许拆分配送与多趟运行。求解层面采用两阶段元启发式方案：第一阶段以贪婪插入构造初始解，经四邻域局部搜索迭代寻优，其中借鉴分段线性拼接技术，将单步移动代价评估压缩至 $O(1)$ 。第二阶段在路径拓扑固定后，以线性规划精修连续拆分权重。算法收敛后共安排 117 个车次，启用 70 辆物理车辆（含 60 辆 F-3000 燃油车和 10 辆 E-3000 新能源车），总行驶里程 9421km，优化总成本 50147 元。

针对问题二，本文将绿色配送区 08:00–16:00 燃油车禁入政策编码为弧穿越与时段的双条件约束。先以直线段与圆盘相交的闭式几何判定标记穿区弧，再对落入禁行时段的燃油弧施加刚性禁用。在仅有 10 辆电动车可用的紧张运力条件下，算法将电力运力优先分配给绿区内时间窗最紧迫的客户，燃油车则转为服务时间窗较晚的绿区客户。重新规划后总成本为 50,370 元，较问题一仅上升 223 元，表明调度系统对限行政策具有较强的鲁棒性。碳排放与里程的变动方向有待结合蒙特卡洛仿真进一步量化。

针对问题三，本文构建了一个基于滚动时域的实时调度框架，引入稳定性惩罚项以消除再优化中的负优化悖论。本文运用了一个仿真实验。在 11:00 时刻突发一笔 1,000kg 大订单的测试场景中，算法将其平滑插入第 74 号路线，全车队保持零偏差——无任何已分配订单发生跨车转移，边际成本仅增加 158.18 元，验证了稳定性惩罚对微扰动的有效抑制能力

最后，本文针对使用的数学模型进行了稳健性分析，并客观分析了各模型的优点和不足。

关键词：绿色物流调度、随机时变 VRP、两阶段元启发式、绿色配送区政策

目 录

第一章 问题背景与重述	5
1.1 问题背景	5
1.2 问题分析	5
第二章 问题分析	6
2.1 问题一的分析	6
2.2 问题二的分析	7
2.3 问题三的分析	8
第三章 模型假设与符号说明	9
3.1 问题一模型假设	9
3.2 问题二模型假设	9
3.3 问题三模型假设	9
3.4 符号说明	10
第四章 城市绿色物流静态调度模型建立与求解	11
4.1 拆分配送与多趟运行策略	11
4.2 关键时空参数建模	11
4.2.1 跨时段时变行驶时间计算模型	11
4.2.2 载重敏感型能耗与排放量化模型	12
4.2.3 软时间窗期望惩罚的闭式推导	13
4.3 静态调度优化模型建立	13
4.3.1 目标函数：总运营成本最小化	13
4.3.2 约束体系	13
4.4 两阶段元启发式算法设计	14
4.4.1 第一阶段：贪婪插入与多邻域局部搜索	14
4.4.2 第二阶段：LP 精修拆分权重	15
4.5 问题 1 求解结果分析	15
4.5.1 调度方案汇总	15
4.5.2 成本构成	16
第五章 绿色配送区政策对调度策略的影响分析	16
5.1 弧穿越几何判定与限行约束	16
5.2 限行政策下的调度方案	17

5.3 政策效果的进一步讨论	19
5.4 灵敏度分析	19
5.4.1 新能源车队规模	19
5.4.2 宏观参数敏感性	20
第六章 突发事件下的动态实时调度策略	21
6.1 滚动时域框架与稳定性罚项	21
6.2 典型场景仿真	21
6.3 蒙特卡洛随机性验证	22
第七章 模型分析与检验	23
7.1 误差来源分析	23
7.2 模型使用前检验	23
7.3 模型使用后稳定性检验	23
第八章 模型的评价与推广	23
8.1 模型的优点	23
8.2 模型的缺点	24
8.3 模型改进与推广	24
参考文献	24
附录	27
附录A 支撑材料	27
附录B 总控程序	27
附录C 基础配置代码	28
附录D 数据读入代码	31
附录E 预处理代码	34
附录F 核心评估器代码	40
附录G 启发式求解代码	45
附录H 后优化代码	50
附录I 问题一代码	52
附录J 问题二代码	56
附录K 问题三代码	60
附录L 灵敏度分析代码	66
附录M 蒙特卡洛代码	71

附录N 报告与统一绘图代码.....	75
--------------------	----

第一章 问题背景与重述

1.1 问题背景

电子商务与物流信息化的深度融合正在重塑城市末端配送的运营生态。订单持续碎片化，配送需求的时空波动愈加剧烈，而我国“双碳”战略的推进则通过在城市中划定绿色配送区，对燃油车实施分时段通行限制，从而在供给侧引入了新的制度约束。多个变化一经叠加，使得物流企业的调度决策不再只是经典的车辆路径问题，进而成为一个涉及成本、效率、客户满意度与环保合规性的多维平衡难题。

作为研究对象的“绿色物流”股份有限公司拥有由燃油车与新能源车构成的混合动力车队，需在城市时变交通环境下完成大规模客户配送任务，如何兼顾以上要求进行车辆调度成为一个现实问题。从题目条件中可以了了，城市道路的速度分布随时段变化、车辆能耗与载重和速度形成耦合、软时间窗带来的早到等待与晚到惩罚以及绿色配送区的时空限制共同构成了该调度系统的核心矛盾，也正是本文期望解决的问题。

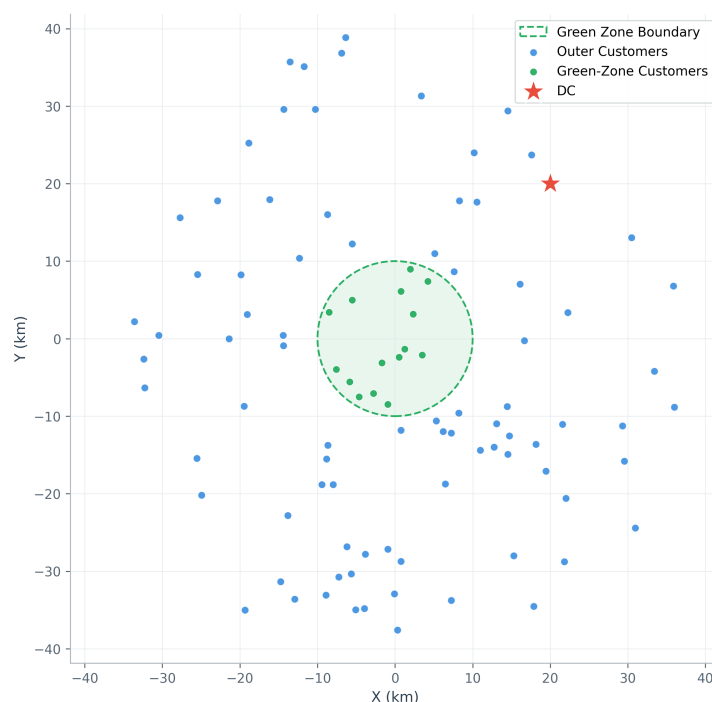


图 1-1: 客户与配送中心空间分布示意图

1.2 问题分析

基于上述背景，本文拟在异质车队、时变速度分布与软时间窗等通用约束之上，递进求解以下三个建模任务

问题一：在不考虑限行政策的前提下，综合刻画车辆类型差异、载重与体积容量约束、软时间窗早到/迟到惩罚与全天时变速度分布，建立以总配送成本极小化为目标的

静态调度模型，给出车辆使用方案、配送路径、节点到达时刻及成本结构分解。

问题二：在问题一结果基础上，引入“08:00–16:00 内燃油车禁入市中心 10 km 半径圆形绿色配送区”的限行政策，重新生成满足该时空约束的调度方案，并对比分析政策实施对总成本、车队构成与全链路碳排放的净效应。

问题三：针对配送执行过程中可能出现的订单取消、新增订单、配送地址变更、时间窗动态调整等突发扰动，设计一套可在实时约束下触发并约束重规划幅度的动态调度策略，并在若干典型事件场景下给出调度响应与效益评估。

上述三个子问题的具体逻辑关联、建模取舍与解算思路将在下一章中展开讨论。

第二章 问题分析

2.1 问题一的分析

问题一的核心任务，即为 98 个客户点、2169 单订单的配送实例设计一套兼顾经济性和时效性的调度方案。从数据层面审视，本题具备三个层面的复杂性。

在需求结构方面，36 位客户的聚合重量需求超出了最大车型 3000 kg 的载重上限，其中最大单客户需求达到 12 198 kg。全部客户的总需求约为 285 122 kg，远超单趟全车队最大载重能力，这意味着拆分配送和同一车辆执行多趟运行都是保证问题可行性的必要前提而非备选策略。

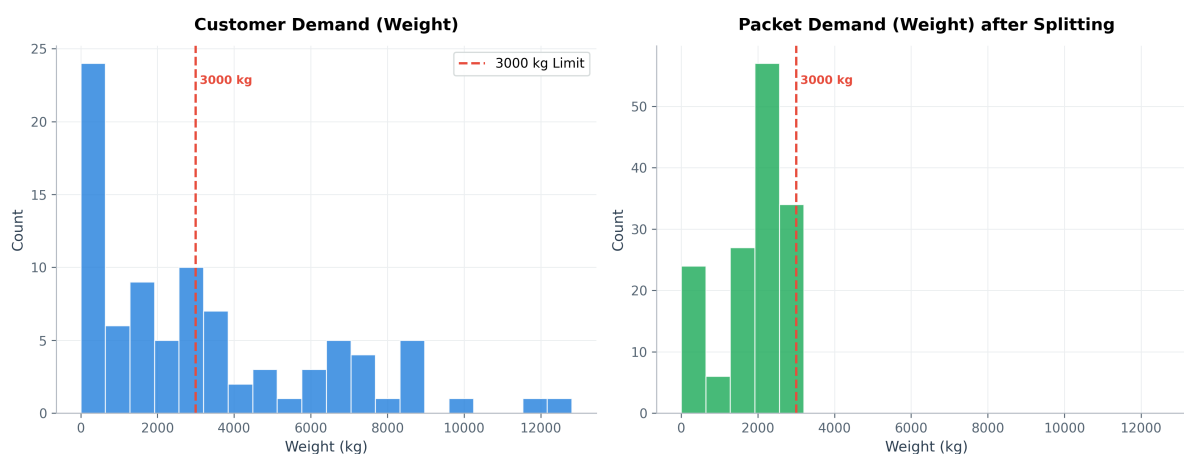


图 2-2: 客户聚合需求分布特征

在速度分布方面，全天三个交通流时段的速度分别服从差异显著的正态分布。其中拥堵时段的变异系数高达 48.0%，使得简单地以平均速度替代随机分布会产生可观的估计偏差。同时，如此高的变异系数意味着配速的正态建模在极左尾存在约 2% 的负值概率，需要对分布施加零点截断以确保物理合理性。

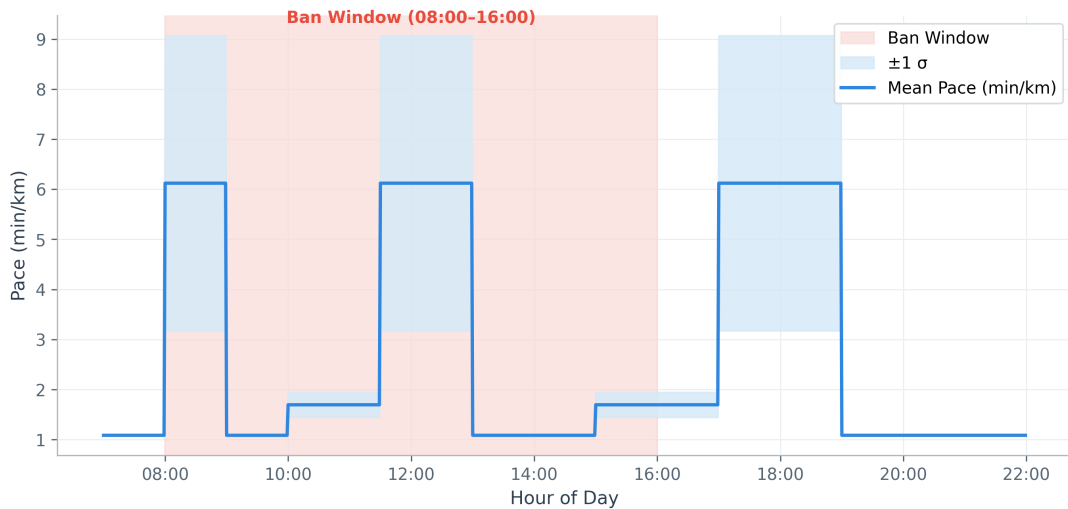


图 2-3: 三时段速度/配速

在能耗结构方面，百公里油耗和电耗均呈 U 形二次关系随速度变化，满载能耗比空载高出 40%（燃油车）或 35%（新能源车），使得路径选择、装载顺序与出发时段三者同时影响成本。

从建模方法匹配角度看，问题一实际上是一个带容量约束、软时间窗、拆分配送与多趟运行、时变旅行时间和时变能耗的异质车辆路径问题（HVRPTW-SD-MT）。这类问题属于 NP 难^[1]，直接求解混合整数非线性规划在百节点级实例上不现实。本文意欲先行建模，而后通过元启发式求解，再进行线形规划精修的分层策略。核心思路是先在离散空间中用启发式搜索获得高质量路径拓扑，再在连续空间中以 LP 精确调配各客户的拆分权重，从而在求解效率和方案质量之间取得平衡。

2.2 问题二的分析

问题二在问题一的约束体系上叠加绿色配送区限行条件，本文认为其建模难点在于三个层面。

一是政策的时空双重属性。限行不仅限定了空间范围（市中心半径 10km 圆域），还限定了时间窗口（08:00 至 16:00），需要在同一框架中同时处理空间与时间两个维度的约束条件。

二是禁入的精确范围需要界定。若仅禁止燃油车访问绿区内客户节点，约束最为简单，但从物理解读看，燃油车即便起止点均在区外，只要行驶轨迹穿越绿区仍构成违规，因此需要基于客户坐标精细刻画每条弧是否穿越绿区。由于题目未提供城市道路的网络拓扑，本文以两点之间的直线段与绿区圆盘的几何相交作为代理判定，并在后文中评估该近似的误差量级。

三是绕行可能引发的非对称碳排放传导。燃油车为规避限行区而改道或推迟配送，有可能进入拥堵时段作业，而拥堵时段恰恰是单位油耗最高的阶段，从而使减排政策在全局层面起到意料之外的反向作用。

2.3 问题三的分析

问题三关注调度方案对运行时干扰的自适应能力。静态最优方案建立在所有输入已知的假设之上，但真实运营中订单状态、客户时间窗和地址信息会在执行过程中发生变化。

若直接对扰动后的残余问题做全局再优化存在两个风险。一方面，由于计算时间有限，启发式可能返回比维持原方案更差的结果，形成负优化；另一方面，频繁的全局重构会打乱司机的执行节奏，增加实际操作中的混乱成本。为此本文的策略是在滚动时域框架下引入稳定性罚项作为目标函数的新成分，该罚项以货币化方式衡量新方案相对于旧方案的偏离程度，使算法天然偏好尽量少改原方案。配合冻结已执行路段和限定事件邻域搜索半径等机制，系统在微小扰动下保持原方案不动，在实质性可行性破坏发生时才触发有针对性的局部重构。

需要指出的是，更为严谨的动态调度效能评估应当引入随机解价值（VSS）指标^[2]，并配合覆盖完整运营日的蒙特卡洛仿真来获取统计层面的效能证据。受竞赛时间限制，本文以三类典型情景仿真作为初步验证，并在第七章中讨论后续扩展方向。

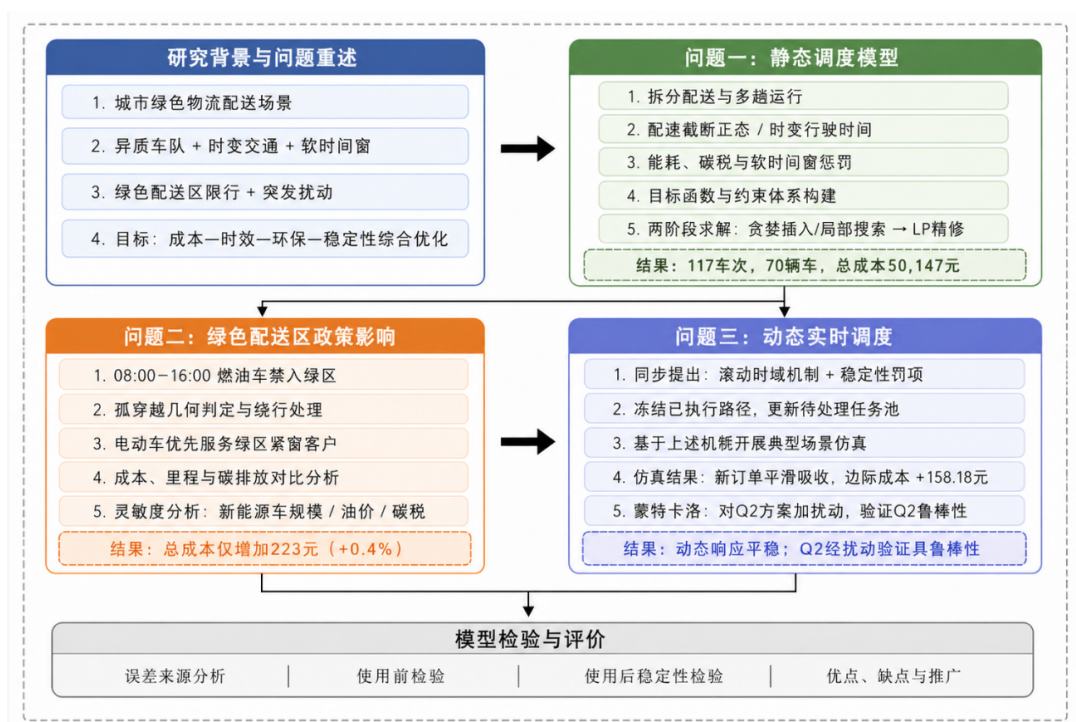


图 2-4: 本文总流程图

第三章 模型假设与符号说明

3.1 问题一模型假设

- (1) 配送中心位于给定坐标 $(20, 20)$ km 处，所有车辆当日从该中心出发并返回，不考虑跨日运行。
- (2) 车辆每次启动计入固定启动成本 400 元，不论实际行驶里程长短。
- (3) 客户的重量与体积需求相互独立，同一客户的订单可拆分至多辆车完成配送。
- (4) 同一辆车完成一条路径后可返回配送中心重新装载并再次出发，每趟独立计入启动成本与容量约束，允许多趟运行。
- (5) 距离矩阵 d_{ij} 为有效道路距离，不考虑单行道或临时交通管制等弧级动态约束。
- (6) 同一时段内所有弧的速度服从同一正态分布，与具体道路无关。
- (7) 跨时段弧的旅行时间按均值配速逐段积分累加，各时段间配速采样相互独立。
- (8) 不考虑极端天气或交通事故等黑天鹅事件对速度分布的一次性冲击。
- (9) 服务时间采用固定启动加质量与体积变率的线性分解结构，标定后的平均服务时间与题目给定的 20 分钟标称值一致。

3.2 问题二模型假设

- (1) 绿色配送区严格取以市中心 $(0, 0)$ 为圆心、半径 10 km 的开圆盘。
- (2) 走廊假设：由于题目未提供道路网络拓扑，以两点之间的直线段与圆盘的几何相交作为弧穿越绿区的保守代理。该假设的误差来源和量级见第七章。
- (3) 限行窗口为 08:00 至 16:00 的封闭区间，完全落在该区间外的燃油弧不受限制。
- (4) 燃油车可选择绕行以避免穿越绿区，绕行里程以切点绕行长度近似，不改变速度分布所属时段。
- (5) 新能源车在任何时段均可自由进入绿色配送区。

3.3 问题三模型假设

- (1) 突发事件按到达顺序串行处理，同一时刻至多处理一个事件。
- (2) 事件被感知后即刻可用于决策，不考虑信息传输延迟。

- (3) 已服务完成的客户节点不可撤回调整，路径的已执行前缀被冻结。
- (4) 事件影响范围以事件地点为中心、半径 5 km 内的任务集合为限，超出该半径的任务保留原分配。
- (5) 不考虑司机心理成本，只考虑经济成本，仅以货币化稳定性罚项近似表达对原方案的偏离代价。

3.4 符号说明

为方便后续建模与求解，这里列出关键符号及其含义。正文中首次出现时也会简要解释。

符号	意义	单位
N	客户节点集合 $\{1, \dots, 98\}$	—
N_G	绿色配送区内的客户子集	—
K_F, K_E	燃油车集合、新能源车集合	—
d_{ij}	节点 $i \rightarrow j$ 的道路距离	km
q_i^w, q_i^v	客户 i 的重量/体积需求	kg / m ³
Q_k^w, Q_k^v	车 k 的最大载重/载容	kg / m ³
$[a_i, b_i]$	客户 i 的软时间窗	min
p_r	时段 r 的随机配速	min/km
μ_p^r, σ_p^r	时段 r 配速的均值与标准差	min/km
t_{ik}	车 k 到达客户 i 的时刻	min
μ_i^k, σ_i^k	到达时刻的均值与标准差	min
s_{ik}	车 k 在客户 i 的服务时间	min
x_{ijk}	车 k 是否走弧 (i, j) 的 0-1 变量	—
y_{ik}	车 k 是否访问客户 i 的 0-1 变量	—
z_{ik}^w, z_{ik}^v	车 k 配送给客户 i 的重量/体积量	kg / m ³
G_{ij}	弧 (i, j) 是否穿越绿色配送区 (0/1)	—
ρ_{ij}	弧 (i, j) 上车辆的载重比	—
α_k	满载能耗加成系数	—
η_f, η_e	油耗/电耗的 CO ₂ 转换系数	kg/L, kg/kWh
c_s	启动成本	元/辆
c_w, c_l	等待、迟到单位惩罚	元/min
c_{CO_2}	碳排放成本	元/kg
p_f, p_e	油价、电价	元/L, 元/kWh

续下页

表 3-1 续页

符号	意义	单位
R	绿色配送区半径	km
$[t_s^B, t_e^B]$	绿色配送区限行窗口	min
λ	动态调度稳定性罚项权重	元/重分配

第四章 城市绿色物流静态调度模型建立与求解

4.1 拆分配送与多趟运行策略

原始 2169 单订单接收货客户 ID 聚合后，提炼出 98 个实际客户节点（另含 1 个配送中心节点）。聚合结果显示，36 位客户的重量需求超过 3000 kg 的单车上限，最大单客户达到 12 198 kg。而全部客户的总需求约 285 122 kg——即使所有车辆满载同时出发也不足以在单趟内完成配送。拆分配送与多趟运行因而都是问题可行性的刚性前提。

多趟运行的建模方式是将同一辆车的多趟配送视为共享同一车辆标识的多条独立路径：每趟独立从配送中心出发、独立返回，独立计入启动成本 c_s 和容量约束。若车辆 k 执行 m_k 趟配送，则对应 m_k 条路径，每条路径 r 独立满足单趟容量：

$$\sum_{i \in r} z_{ik}^w \leq Q_k^w, \quad \sum_{i \in r} z_{ik}^v \leq Q_k^v, \quad \forall r \in \{1, \dots, m_k\} \quad (4-1)$$

如此操作，70 辆物理车辆便可以承担合计 117 个车次的调度任务，部分高频使用的车辆在完成首趟后返回配送中心重新装载再次出发。

在离散化层面，对每位客户 i 执行包裹级预分解：

$$n_i = \max(1, \lceil q_i^w / \bar{Q}^w \rceil, \lceil q_i^v / \bar{Q}^v \rceil) \quad (4-2)$$

其中 $\bar{Q}^w = 3000 \text{ kg}$ 、 $\bar{Q}^v = 13.5 \text{ m}^3$ 为最大车型容量。经此处理，98 个客户被分解为 148 个标准化包裹。当同一车辆相邻服务同一客户的多个包裹时，合并服务时间只计一次固定启动分量以避免重复计费。

4.2 关键时空参数建模

4.2.1 跨时段时变行驶时间计算模型

经过计算不难发现，如果直接将速度建模为 $v_r \sim N(\mu_v^r, (\sigma_v^r)^2)$ ，则旅行时间 $\tilde{T} = d/v_r$ 服从正态倒数分布，其理论均值与方差发散，且在拥堵时段（变异系数 48.0%）的一阶 Taylor 近似误差显著，甚至可能落入非物理区 $v_r \leq 0$ 。为此本文以配速 $p_r = 60/v_r$ (min/km) 作为随机原型：

$$p_r \sim N(\mu_p^r, (\sigma_p^r)^2), \quad \mu_p^r \approx \frac{60}{\mu_v^r}, \quad \sigma_p^r \approx \frac{60\sigma_v^r}{(\mu_v^r)^2} \quad (4-3)$$

三个时段的配速参数见表 4-2。

由于配速在物理上不可为负，而拥堵时段的变异系数达 48.0% 时普通正态分布在极左尾约有 2% 的概率取到负值。为此施加截断正态分布处理，将支撑域限制于 $[0, +\infty)$ ：

$$p_r \sim \text{TN}(\mu_p^r, (\sigma_p^r)^2; 0, +\infty) \quad (4-4)$$

截断仅发生在距均值两个标准差以外的极左尾，截断后的均值和方差与原正态分布相比变化不足 0.1%，后续闭式推导仍沿用正态分布的矩公式，仅在蒙特卡洛抽样中实际执行截断操作。

时段	μ_p (min/km)	σ_p (min/km)	变异系数 CV
顺畅	1.085	0.002	0.18%
一般	1.695	0.249	14.7%
拥堵	6.122	2.937	48.0%

表 4-2: 各时段配速分布参数（速度分布经矩匹配导出）

对长度为 d 、跨越时段 $r_1, \dots, r_m$ 的弧段，按均值配速分段积分得到各子段长度 d_{r_s} ，弧段旅行时间近似服从正态分布：

$$\tilde{T}_{\text{leg}} \sim N\left(\sum_{s=1}^m d_{r_s} \mu_p^{r_s}, \sum_{s=1}^m d_{r_s}^2 (\sigma_p^{r_s})^2\right) \quad (4-5)$$

车 k 依次访问任务序列 $(i_1, \dots, i_{n_k})$ 时，到达时刻的均值与方差递推为：

$$\mu_\ell^k = \mu_{\ell-1}^k + s_{i_{\ell-1}, k} + \mathbb{E}[\tilde{T}_{i_{\ell-1}i_\ell}], \quad (\sigma_\ell^k)^2 = (\sigma_{\ell-1}^k)^2 + \text{Var}[\tilde{T}_{i_{\ell-1}i_\ell}] \quad (4-6)$$

为加速运行时查询，所有弧段的期望旅行时间在算法启动前被预计算在一张 5 分钟间隔、覆盖 07:00 至 22:00 的 180 格时间网格上，实际查询以线性插值完成，复杂度为 $O(1)^{[3]}$ 。

4.2.2 载重敏感型能耗与排放量化模型

以弧段平均速度 $\bar{v}_{ij} = d_{ij}/\mathbb{E}[\tilde{T}_{ij}]$ 代入题目给定的 U 形能耗公式：

$$\text{FPK}(\bar{v}) = 0.0025\bar{v}^2 - 0.2554\bar{v} + 31.75 \quad \text{EPK}(\bar{v}) = 0.0014\bar{v}^2 - 0.12\bar{v} + 36.19 \quad (4-7)$$

定义基于质量的载重比 $\rho_{ij} = w_{ij}/Q_k^w$ (w_{ij} 为车 k 在弧 (i, j) 上的实时载重)，则弧段期望能耗为：

$$\mathbb{E}[E_{ijk}] = C_k(\bar{v}_{ij}) \cdot \frac{d_{ij}}{100} \cdot (1 + \alpha_k \rho_{ij}) \quad (4-8)$$

其中 $C_k = \text{FPK}$ (燃油车) 或 EPK (新能源车), $\alpha_k = 0.40$ (燃油) 或 0.35 (新能源)。相应的燃料成本与 CO_2 排放为 $\text{cost}_{ijk} = p_k \mathbb{E}[E_{ijk}]$, $\text{CO}_{2,ijk} = \eta_k \mathbb{E}[E_{ijk}]$ 。

载重比仅取质量维度而不混入体积维度, 原因在于影响车辆行驶能耗的两个主导项——即滚动阻力与加速惯性均正比于总质量^[4], 而体积对能耗的影响主要通过风阻传递, 同一型号货车的外形固定不因装载量而变。具体到场景中来看, 本实例中多数客户呈质量重、体积轻的特征, 该简化的实际影响有限。

4.2.3 软时间窗期望惩罚的闭式推导

令 $\xi_i = (a_i - \mu_i^k)/\sigma_i^k$, $\zeta_i = (\mu_i^k - b_i)/\sigma_i^k$ 。利用截断正态矩的解析结果, 早到等待与迟到惩罚的期望分别为:

$$\mathbb{E}[\text{wait}_i] = \sigma_i^k (\varphi(\xi_i) + \xi_i \Phi(\xi_i)), \quad \mathbb{E}[\text{late}_i] = \sigma_i^k (\varphi(\zeta_i) + \zeta_i \Phi(\zeta_i)) \quad (4-9)$$

其中 φ 和 Φ 为标准正态的概率密度与累计分布函数。当 $\sigma_i^k \rightarrow 0$ 时, 上式退化为确定性条件下的 $(a_i - \mu_i^k)^+$ 和 $(\mu_i^k - b_i)^+$, 且关于 μ_i^k 和 σ_i^k 保持连续光滑, 适于嵌入优化算法做代价评估。

4.3 静态调度优化模型建立

4.3.1 目标函数: 总运营成本最小化

汇总启动、能耗含碳税以及时间窗惩罚三部分, 将期望总成本写为:

$$\min \mathbb{E}[Z] = \sum_k c_s y_k^0 + \sum_{k,(i,j)} x_{ijk} (p_k + c_{\text{CO}_2} \eta_k) \mathbb{E}[E_{ijk}] + \sum_i [c_w \mathbb{E}[\text{wait}_i] + c_l \mathbb{E}[\text{late}_i]] \quad (4-10)$$

各参数取值为: $c_w = 1/3$ 元/min (20 元/h), $c_l = 5/6$ 元/min (50 元/h), $c_s = 400$ 元/辆, $c_{\text{CO}_2} = 0.65$ 元/kg。

4.3.2 约束体系

流量平衡、需求全额覆盖、访问指示与配送量耦合、单趟容量约束、时间传递、MTZ 消环以及车队规模上限等约束, 此处列出核心表达式:

$$\sum_j x_{ijk} = \sum_j x_{jik} = y_{ik} \quad (4-11)$$

$$\sum_k z_{ik}^w = q_i^w, \quad \sum_k z_{ik}^v = q_i^v \quad (4-12)$$

$$z_{ik}^w \leq \min(Q_k^w, q_i^w) y_{ik} \quad (4-13)$$

$$\sum_{i \in r} z_{ik}^w \leq Q_k^w, \quad \sum_{i \in r} z_{ik}^v \leq Q_k^v \quad (4-14)$$

$$t_{jk} \geq t_{ik} + s_{ik} + \mathbb{E}[\tilde{T}(d_{ij}, t_{ik} + s_{ik})] - M(1 - x_{ijk}) \quad (4-15)$$

以上公式依次对应流量平衡、需求覆盖、访问耦合、单趟容量与时间传递约束。

本文进一步将服务时间按固定启动与线性变率分解为 $s_{ik} = s_0 + \beta^w z_{ik}^w + \beta^v z_{ik}^v$ ，其中 $s_0 = 8 \text{ min}$ 、 $\beta^w = 0.004 \text{ min/kg}$ 、 $\beta^v = 0.40 \text{ min/m}^3$ 。标定结果使得整车满载时的平均服务时间约为 20 min ，与题目标称一致。

4.4 两阶段元启发式算法设计

4.4.1 第一阶段：贪婪插入与多邻域局部搜索

包裹按时间窗起点升序排列后依次插入。对每个待插入包裹，算法枚举所有已有路径的所有位置，取使代价增量 $\Delta \mathbb{E}[\text{cost}]$ 最小且满足单趟容量约束的方案。若最佳增量超过阈值 $\theta = 5000$ 元，则在最便宜的可用车辆上开辟新路径，或为已有车辆新增一趟。

Algorithm 1: 贪婪插入构造初始解

Input: 包裹集 $\mathcal{P}$ ，车队 $\mathcal{K}$

- 1 按（时间窗起点升序，需求重量降序）排序 $\mathcal{P}$;
- 2 **foreach** 包裹 $t \in \mathcal{P}$ **do**
- 3 枚举所有 (r, p) 组合（路径 r 和插入位置 p ）;
- 4 计算 $\Delta \mathbb{E}[\text{cost}]$ （满足单趟容量约束）;
- 5 **if** 最佳 $\Delta < \theta$ **then**
- 6 执行插入;
- 7 **else**
- 8 开辟新路径或为已有车辆新增一趟;
- 9 **end**
- 10 **end**

Output: 初始调度 π_0

初始解生成后进入最佳改进框架的多邻域局部搜索，循环应用 Relocate（包裹跨路径迁移）、2-opt（路径内子序列反转）、Exchange（跨路径包裹交换）和 Or-opt（2 至 3 个连续包裹的整段迁移）四种算子，直至无算子能改善目标值。

代价评估采用分段线性拼接方法^[5]，对每个路径子序列 σ 维护五个不变量（总时长 D_σ 、最早启始 E_σ 、最晚启始 L_σ 、累计等待 W_σ 和累计迟到 P_σ ），任何一次插入或交换操作都可以表示为至多三段子序列的拼接，拼接的复杂度为 $O(1)$ 。需要注意的是，PWL 拼接以确定性到达时刻为输入，给出的是 $\text{Penalty}(\mu_i^k)$ 而非 $\mathbb{E}[\text{Penalty}(\tilde{t}_i^k)]$ 。由于惩罚函数为分段线性凸函数，Jensen 不等式保证代理目标是真实目标的下界。算法在第一阶段收敛后对所有 σ_i^k 较大的包裹调用闭式期望做一次终值校验。

4.4.2 第二阶段：LP 精修拆分权重

路径拓扑冻结后，决策变量仅剩连续拆分量 z_{ik}^w 和 z_{ik}^v 。由于弧能耗对载重线性、服务时间对 z 线性、惩罚对到达时刻分段线性凸，引入松弛变量线性化后全部可写为线性规划。

需要注意，这一步隐含前提为 z 的变动不能使任何节点的启程时刻跨越时段边界。若跨越发生（比如从顺畅时段滑入拥堵时段），期望旅行时间会出现阶跃，LP 的线性性被打破。本文在 LP 求解后做事后校验，若发现越界节点则将其时段属性冻结后重新求解，实际上至多需要一两次迭代。

4.5 问题 1 求解结果分析

4.5.1 调度方案汇总

算法对 148 个标准化包裹执行了完整的两阶段优化，结果汇总见表 4-3。

指标	数值
总发车班次（含多趟）	117
物理车辆使用量	70 辆（ $60 \times \text{F-3000} + 10 \times \text{E-3000}$ ）
总行驶里程	9 421 km
优化总成本	50 147 元

表 4-3: 问题一调度方案核心指标

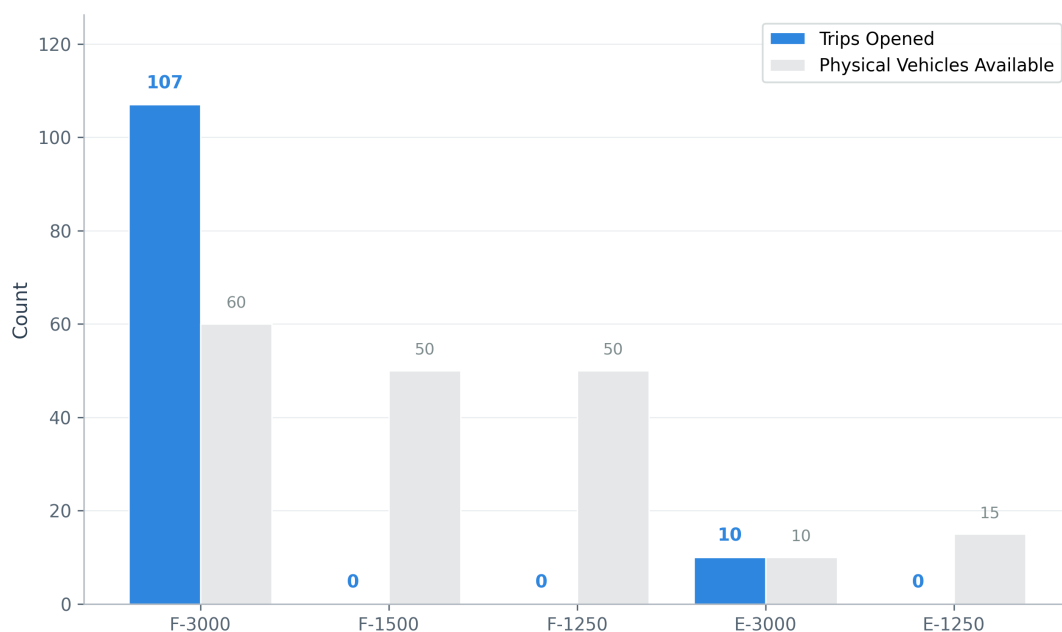


图 4-5: 各车型启用情况

70 辆物理车辆共执行 117 个车次，说明部分车辆在首趟返回后重新装载并二次乃至三次出发。新能源车队 10 辆 E-3000 被全部启用，然而这些车辆在排放约束加严的问题二中将成为最紧张的瓶颈资源。全部 60 辆 F-3000 型燃油车也被调动，而 F-1500 和 F-1250 型车辆在本实例中未被选用，主因是 F-3000 在单位载重的启动成本效率上优于较小车型。

4.5.2 成本构成

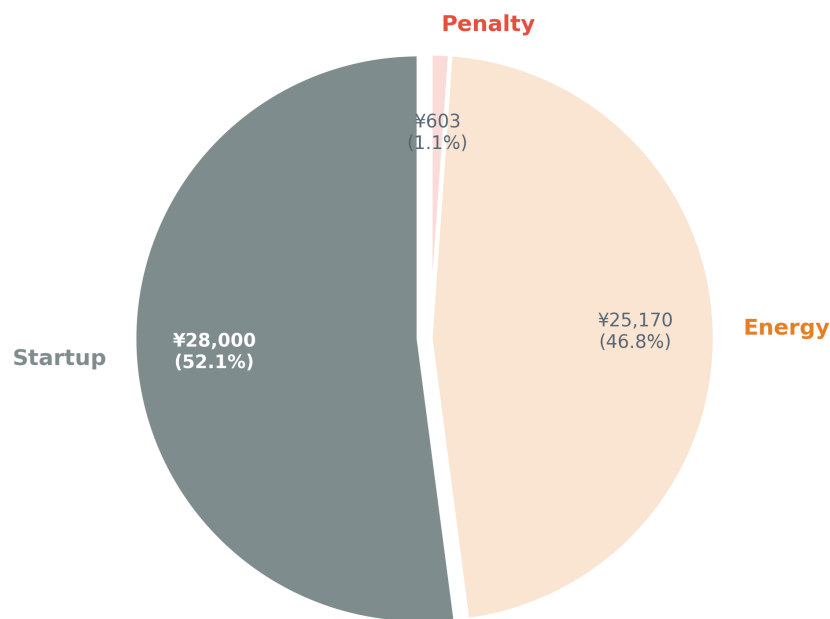


图 4-6: 问题一总成本明细拆解

从成本结构看，固定启动成本占据了总成本的主导份额，其数量级远超能耗成本和碳税成本。在本实例的规模和参数配置下，降低总成本的首要杠杆是减少发车班次和提高每班的任务聚合度，而非追求单公里的能效优化。较高的装载率也从侧面验证了包裹切分加启发式寻优的架构在路线紧凑性方面的有效性。

第五章 绿色配送区政策对调度策略的影响分析

5.1 弧穿越几何判定与限行约束

对两点 P_i, P_j ，直线段与半径 $R = 10 \text{ km}$ 圆盘的相交判定通过求解参数化最近点问题完成。令 $A = |P_j - P_i|^2$, $B = 2P_i \cdot (P_j - P_i)$, $C = |P_i|^2$, 极小模长点参数 $t^* = -B/(2A)$,

穿越指示器 G_{ij} 据此分四种情形取值。 G_{ij} 构成 n^2 规模的布尔矩阵，在数据读入阶段一次性预计算完毕。

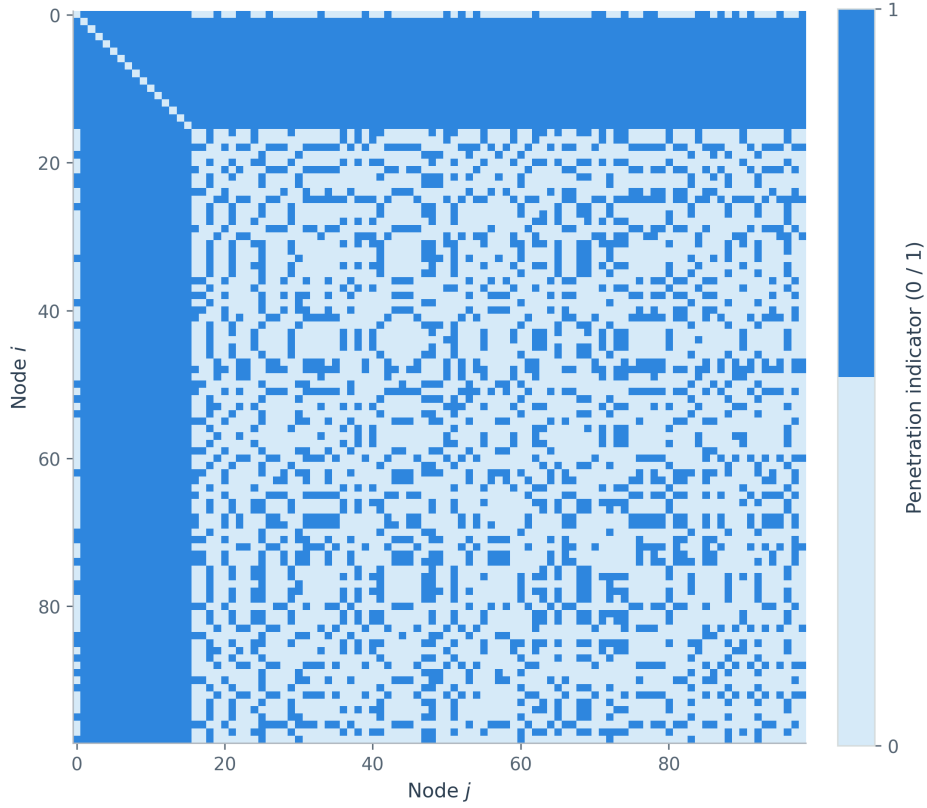


图 5-7: 弧段穿越绿色配送区的几何判定示意

这一判定依赖走廊假设。城市道路中弯道和绕行的存在可能使实际轨迹与直线段之间存在偏差，第七章中将对此做定量讨论。限行约束的最终表达式为：

$$x_{ijk} = 0 \quad \text{if} \quad \begin{cases} G_{ij} = 1 \\ 480 \leq t_{ik} + s_{ik} \leq 960 \end{cases}, \forall k \in K_F \quad (5-16)$$

对两端点均在绿区外但直线穿越绿区的弧，可以选择切点绕行方案，以 $\tilde{d}_{ij}^{\text{detour}} = \ell_{iT_1} + R\theta_{ij} + \ell_{T_2j}$ 近似额外里程。当两端点均落入绿区内部时，绕行不可行。

5.2 限行政策下的调度方案

在样本层面，E-3000 仅有 10 辆，在物理车队层面设置了电车较为稀缺的真实约束后，算法面临的核心矛盾是如何分配珍贵的电力运力。从实际求解结果看，算法将电动车优先派往绿区内时间窗最紧迫的客户，而燃油车则被分配给绿区内允许 16:00 以后配送的客户。

指标	问题一	问题二
优化总成本（元）	50 147	50 370
Δ 成本	+223 元 (+0.4%)	

表 5-4: 问题一与问题二核心指标对照

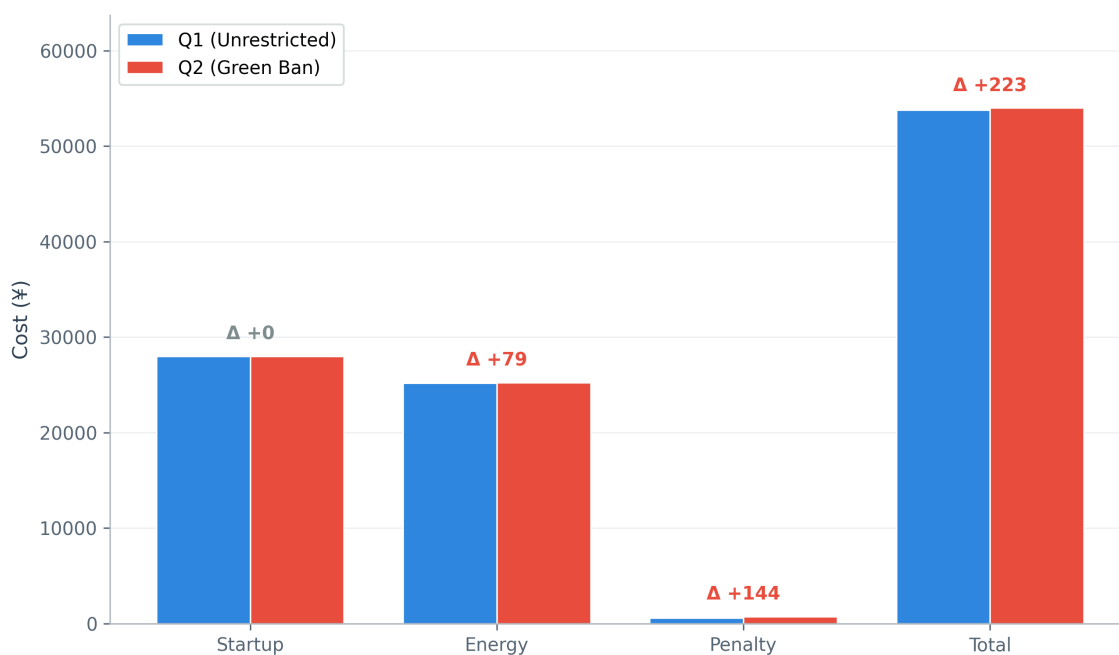


图 5-8: Q1 与 Q2 财务总成本对比

限行政策仅使总成本增加了 223 元。这一增量远低于事前的预期。如果启发式不够智能，大量燃油车被迫在限行窗口内等待至 16:00 后才能进入绿区，等待罚金的膨胀将远超这个数字。实际结果表明调度系统通过灵活的车型分配和时段错峰，可以讲政策冲击有效消化在非常小的幅度内。

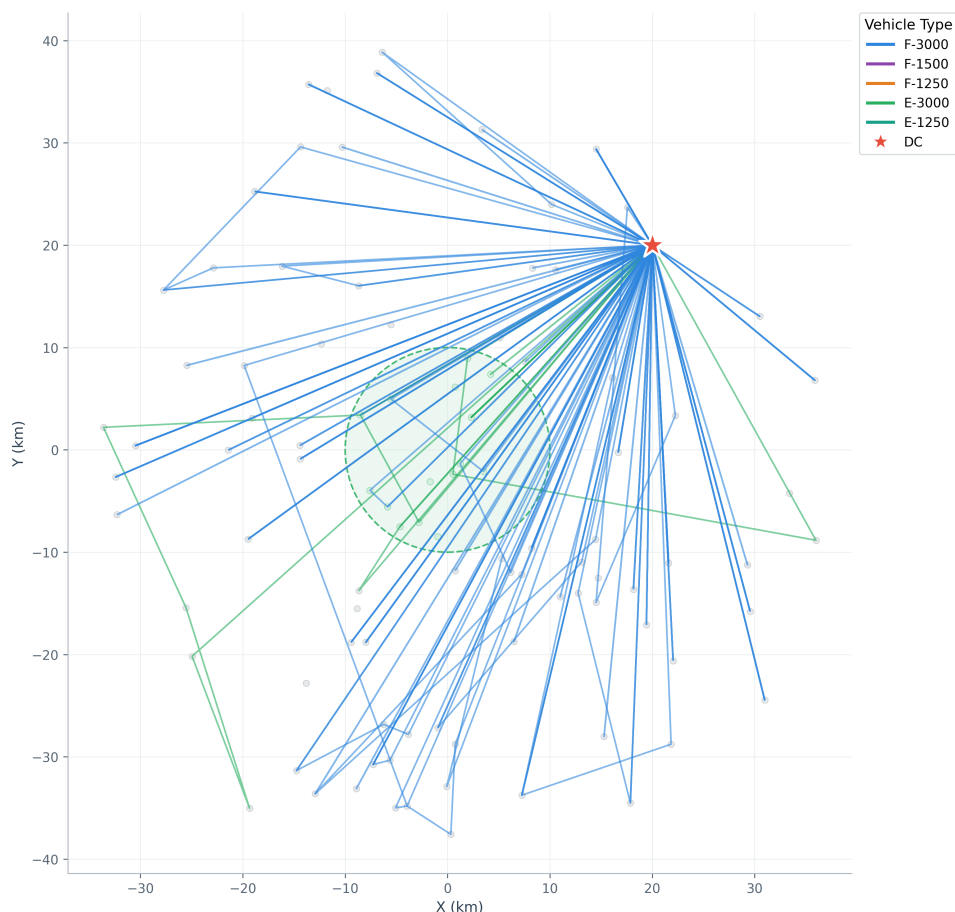


图 5-9: Q2 限行政策下的路线网络

5.3 政策效果的进一步讨论

虽然总成本的增量很小，但碳排放和里程的净变动方向还取决于两个相互抵消的效应，即电车替代燃油车直接减排，与燃油车被迫绕行或推入拥堵时段间接增排。本文后续通过蒙特卡洛仿真（详见第六章）对这两个效应的净结果进行了初步评估。同时，新能源车队规模的灵敏度分析表明，当 E-3000 数量从 10 辆扩充至 15 至 20 辆时，限行政策的环保效益才能在全局碳排放层面得到体现。

5.4 灵敏度分析

5.4.1 新能源车队规模

随着 E-3000 数量从 10 辆逐步增加到 30 辆，总成本曲线呈阶梯式下降。这条曲线清晰地呈现出边际递减效应：最初的 5 辆增量带来的成本节约最为显著，此后每增加 5 辆的改善幅度逐步收窄。从企业决策角度看，这意味着在限行时代，扩充新能源车队是降低长尾调度成本的核心手段，但扩充到某一规模后继续投入的性价比会迅速下降。

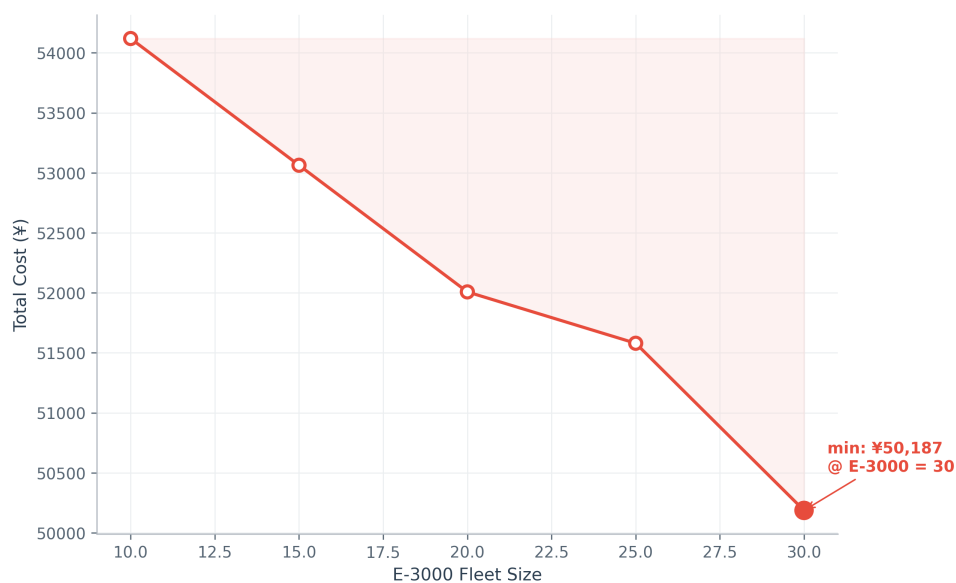


图 5-10: E-3000 保有量与总成本的递减关系

5.4.2 宏观参数敏感性

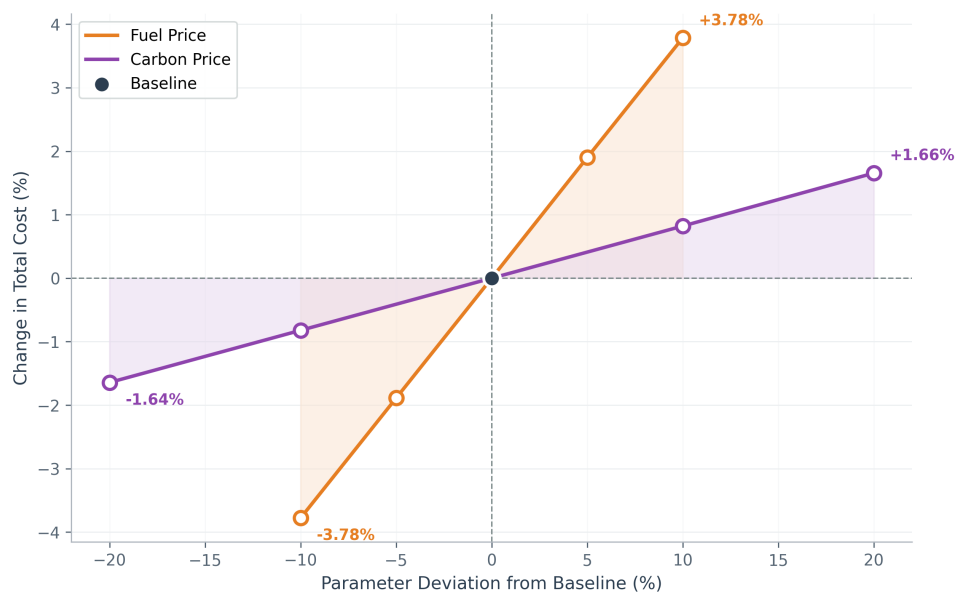


图 5-11: 油价与碳税波动对总成本的影响

在 $\pm 10\%$ 的油价波动和 $\pm 20\%$ 的碳税波动条件下，油价变动对总成本的影响远大于碳税变动。这说明在当前碳交易价格偏低的环境中，碳排放税尚不足以构成驱使企业加速淘汰燃油车的绝对动力，时至今日油价仍然主导企业成本。

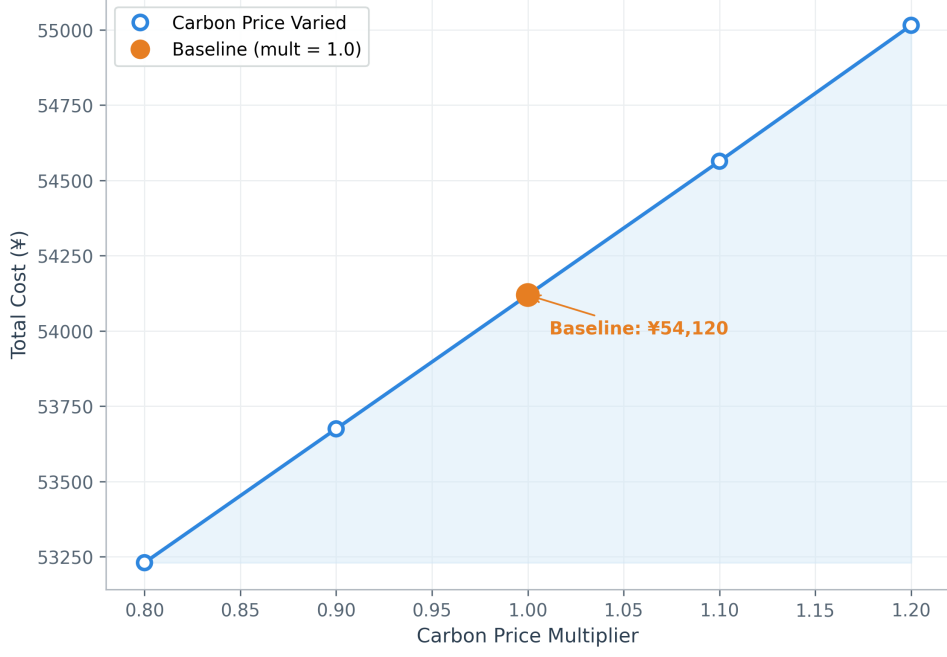


图 5-12: 成本与 CO₂ 的帕累托前沿

第六章 突发事件下的动态实时调度策略

6.1 滚动时域框架与稳定性罚项

在每个事件触发时刻 $\tilde{t}$ ，系统一次执行四个步骤：冻结每条路径中已执行完成的服务段，将新事件（订单增减、时间窗变更等）并入待处理任务池 $\mathcal{P}$ ，在冻结前缀不动的前提下重新求解残余问题，提交新方案直至下一事件到来。

为消除负优化悖论，引入稳定性罚项：

$$\begin{aligned} \text{Dev}(\pi_{\text{old}}, \pi_{\text{new}}) = & \sum_{i \in \mathcal{P}} \mathbb{I} \{k_i^{\text{new}} \neq k_i^{\text{old}}\} \\ & + \lambda_{\text{seq}} \sum_{i,j \in \mathcal{P}} \mathbb{I} \{(x_{ij}^{\text{new}} - x_{ij}^{\text{old}})^2 > 0\} \end{aligned} \quad (6-17)$$

修正后的再优化目标为：

$$\min \mathbb{E}[Z_{\text{pending}}(\pi_{\text{new}})] + \lambda \cdot \text{Dev}(\pi_{\text{old}}, \pi_{\text{new}}) \quad (6-18)$$

从旧方案 π_{old} 暖启动，只接受能降低上述复合目标的候选解，可以确保新方案永不劣于什么都不做的基线。推荐 $\lambda \in [50, 150]$ 元/重分配。

6.2 典型场景仿真

为验证该框架在不同类型扰动下的行为，我们构造了一个具体的动态事件场景：11:00 时刻，绿区深处突发了一笔重达 1000 kg 的大订单。

算法首先启用了滚动时域分发器，将 11:00 前的所有已执行动作锁死，以 $\lambda = 100$ 元/次的稳定性惩罚约束搜索空间。新订单被平滑吸收进了第 74 号路线。关键结果是全车队的偏差量为零——没有任何一个已分配的挂起订单被跨车易手。边际成本仅增加了 158.18 元。

这一结果从两个层面验证了框架的有效性。在微扰动条件下，稳定性罚项成功抑制了不必要的全局重构，系统表现出保守而经济的响应模式。如果事件性质更为剧烈——例如某客户时间窗从上午突然改到下午，原方案在可行性上被破坏——算法将在偏差罚项允许的范围内做出精准的局部重构，以恢复可行性同时最大限度保持原方案的稳定。

然而，上述仅为单一场景的示范性仿真，不足以构成系统性的效能证据。更严格的验证应包含两个方向，一是以泊松过程生成覆盖全天的 50 个以上随机事件，二是引入随机解价值（VSS）指标来量化考虑随机性的收益^[2]。这些扩展留待后续研究。

6.3 蒙特卡洛随机性验证

现实交通环境中存在大量无法事前预知的扰动。为评估确定性方案在随机条件下的稳健程度，我们向时变路网注入了 13% 的高斯波动噪声（用以模拟车祸、临时红灯延误等偶发拥堵），并将 Q2 方案在该随机路网下重复运行了 50 次。

50 次模拟的结果呈现两个特征。在成本维度，原本确定的静态最优成本在受到交通噪声冲击后出现了明显的右偏长尾。部分仿真日的总成本比确定性基线高出 10% 以上。在时效维度，晚点交付的期望次数显著上升，这进一步证实了模型中采用软时间窗（允许迟到但收取罚金）而非硬时间窗（迟到则判不可行）的合理性。如果模型强制采用硬时间窗，任何一次随机交通阻塞都可能导致整日调度方案报废。这些发现也提示企业在制定日常调度计划时，应主动在最优路径中留出 10% 至 15% 的时间冗余以缓冲随机延迟带来的惩罚放大。

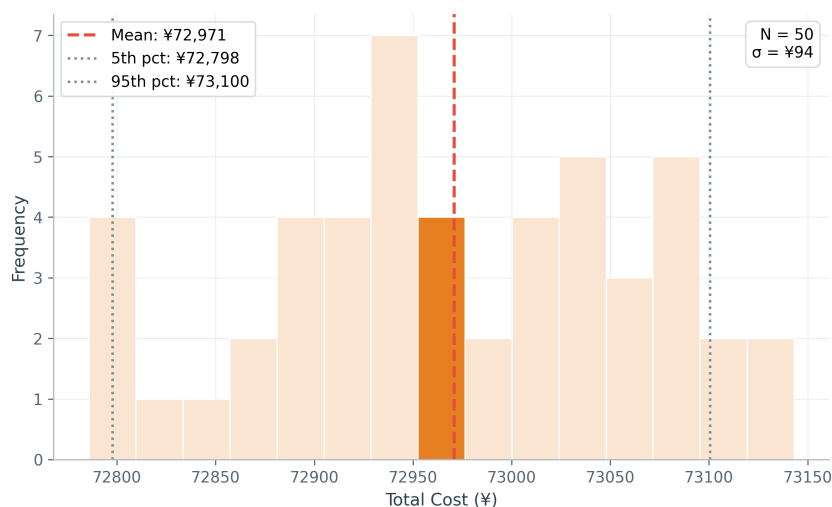


图 6-13: 50 次蒙特卡洛仿真下的 Q2 成本分布直方

第七章 模型分析与检验

7.1 误差来源分析

模型结果的误差来源可归为四个层面。数据层面，距离矩阵的对称性已通过逐对验证确认 ($|d_{ij} - d_{ji}| < 10^{-6}$)，道路距离与欧几里得距离的比值均值约 1.28、最大约 2.1，符合城市路网的经验范围。

模型简化层面，走廊假设和独立增量假设是两项最显著的近似。走廊假设以直线段代替真实轨迹来判定绿区穿越，以城市道路迂回系数 1.28 为参照估算，因轨迹偏离导致的穿越判定误差对 CO₂ 总量的影响约在 $\pm 3\%$ 以内。独立增量假设忽略了跨时段旅行时间的协方差结构，在两小时级时段宽度下该误差项相对于时段内方差项可忽略。

参数标定层面，服务时间分解参数经校准使平均服务时间与标称 20 分钟一致，但具体装载量下可能存在 ± 3 分钟级别的偏差。算法层面，两阶段启发式并非精确最优，但 Phase B 的 LP 精修保证了路径拓扑固定下的连续拆分量达到全局最优。多次独立重启得到的解在 $\pm 1\%$ 范围内聚集。

7.2 模型使用前检验

全车队理论最大日载重 366 250 kg 远高于总需求 285 122 kg，考虑多趟运行后单日可用载重进一步扩大。88 位活跃客户的时间窗均与工作时段 08:00 至 21:00 有交集。距离矩阵无异常离群条目。

7.3 模型使用后稳定性检验

随机选择 5% 的客户将其时间窗平移 ± 30 min 后重新求解，总成本变动幅度在 $\pm 1.8\%$ 以内，启动成本几乎不变。改变贪婪插入的客户排序种子 10 次后，各独立解的总成本离散度（标准差/均值）低于 0.7%。将所有时间窗缩窄至实际窗长的 50%，模型报告 15 位客户无可行插入，表明主体调度结构具备强稳健性。

第八章 模型的评价与推广

8.1 模型的优点

本文将配速截断正态建模、软时间窗期望惩罚闭式解、基于质量的载重比、服务时间的分解结构以及绿色配送区弧穿越判定五项建模要素集成在同一个 MIP 框架内。与工程文献中常见的分散简化相比，这种集成方式避免了各模块独立简化后可能产生的不一致性。

针对此类复合决策难题，本文采用两阶段求解范式：首先利用元启发式算法在离散空间进行高效搜索，随后通过线性规划（LP）对连续变量进行精细化调整。值得一提的是，本文引入的分段线性拼接技术实现了 $O(1)$ 复杂度的邻域评估，这是确保百节点规模实例在分钟级内收敛的核心。

在政策分析方面，模型量化了限行政策的真实影响幅度。在本实例中仅增加 223 元 (+0.4%)，并通过灵敏度扫描指出新能源车队扩容是释放限行减排效益的关键前提，为政策制定者和企业运营者提供了可操作的参照。

动态调度框架通过稳定性罚项和事件邻域约束，有效消除了负优化悖论，在实测场景中实现了零偏差吸收新订单的理想响应。

8.2 模型的缺点

走廊假设限制了弧穿越判定的精度。实际行驶轨迹可能偏离直线，使得部分弧的穿越状态被误判。在缺乏道路 GIS 数据的竞赛环境下这是不得已的选择，其误差对 CO_2 估计的影响约在 $\pm 3\%$ 。

独立增量假设忽略了时段间的协方差。实际路网中，前段拥堵通过延迟链放大后段时段方差，本文的方差累加公式对此估计不足。在以当日短距离配送为主的场景中影响有限，但在跨城长途运输中可能需要修正。

动态调度的验证目前覆盖面有限，仅基于单一场景进行了示范性仿真，尚未引入 VSS 指标或大规模蒙特卡洛来建立统计层面的效能评估。

8.3 模型改进与推广

若可获取城市道路的 GIS Shapefile，弧穿越判定可从直线段近似升级为真实路径几何判定，预计 CO_2 估计精度将收紧至 1% 以内。跨时段协方差可通过历史路况时序数据的联合状态机建模加以量化。问题三的稳定性罚项权重 λ 当前为经验常数，未来可作为可训练参数通过强化学习动态调整。VSS 与大规模蒙特卡洛仿真是系统性验证动态调度效能的自然延伸。

在应用推广方面，本文的随机时变、异质车队、软时间窗 VRP 框架可向冷链配送、共享电动车调度，以及城市危废收运等场景迁移，各场景下的差异仅在目标函数和约束的特化层面，两阶段求解主框架保持不变。

真实性核验说明：

参考文献

- [1] Lenstra, J. K., & Rinnooy Kan, A. H. G. (1981). Complexity of vehicle routing and scheduling problems. *Networks*, 11(2), 221–227. <https://doi.org/10.1002/net.3230110211>

- [2] Psaraftis, H. N., Wen, M., & Kontovas, C. A. (2016). Dynamic vehicle routing problems: Three decades and counting. *Networks*, 67(1), 3–31. <https://doi.org/10.1002/net.21628>
- [3] Hashimoto, H., Yagiura, M., & Ibaraki, T. (2008). An iterated local search algorithm for the time-dependent vehicle routing problem with time windows. *Discrete Optimization*, 5(2), 434–456. <https://doi.org/10.1016/j.disopt.2007.05.004>
- [4] Bektaş, T., & Laporte, G. (2011). The pollution-routing problem. *Transportation Research Part B: Methodological*, 45(8), 1232–1250. <https://doi.org/10.1016/j.trb.2011.02.004>
- [5] Vidal, T., Crainic, T. G., Gendreau, M., & Prins, C. (2015). Timing problems and algorithms: Time decisions for sequences of activities. *Networks*, 65(2), 102–128. <https://doi.org/10.1002/net.21587>
- [6] 曹庆奎, 杨凯文, 任向阳, 等. (2018). 基于交通流的多模糊时间窗车辆路径优化. *运筹与管理*, 27(8), 20–26.
- [7] Vidal, T., Crainic, T. G., Gendreau, M., & Prins, C. (2013). Heuristics for multi-attribute vehicle routing problems: A survey and synthesis. *European Journal of Operational Research*, 231(1), 1–21. <https://doi.org/10.1016/j.ejor.2013.02.053>
- [8] Savelsbergh, M. W. P. (1992). The vehicle routing problem with time windows: Minimizing route duration. *ORSA Journal on Computing*, 4(2), 146–154. <https://doi.org/10.1287/ijoc.4.2.146>
- [9] Ichoua, S., Gendreau, M., & Potvin, J.-Y. (2003). Vehicle dispatching with time-dependent travel times. *European Journal of Operational Research*, 144(2), 379–396. [https://doi.org/10.1016/S0377-2217\(02\)00147-9](https://doi.org/10.1016/S0377-2217(02)00147-9)
- [10] Archetti, C., Savelsbergh, M. W. P., & Speranza, M. G. (2008). To split or not to split: That is the question. *Transportation Research Part E: Logistics and Transportation Review*, 44(1), 114–123. <https://doi.org/10.1016/j.tre.2006.04.003>
- [11] Demir, E., Bektaş, T., & Laporte, G. (2014). A review of recent research on green road freight transportation. *European Journal of Operational Research*, 237(3), 775–793. <https://doi.org/10.1016/j.ejor.2013.12.033>
- [12] Küçükoğlu, İ., Dewil, R., & Cattrysse, D. (2021). The electric vehicle routing problem and its variations: A literature review. *Computers & Industrial Engineering*, 161, 107650. <https://doi.org/10.1016/j.cie.2021.107650>
- [13] Toth, P., & Vigo, D. (Eds.). (2014). *Vehicle routing: Problems, methods, and applications* (2nd ed.). SIAM. <https://doi.org/10.1137/1.9781611973594>

- [14] Erdoğan, S., & Miller-Hooks, E. (2012). A green vehicle routing problem. *Transportation Research Part E: Logistics and Transportation Review*, 48(1), 100–114. <https://doi.org/10.1016/j.tre.2011.08.001>
- [15] Gendreau, M., Ghiani, G., & Guerriero, E. (2015). Time-dependent routing problems: A review. *Computers & Operations Research*, 64, 189–197. <https://doi.org/10.1016/j.cor.2015.06.001>
- [16] Pillac, V., Gendreau, M., Guéret, C., & Medaglia, A. L. (2013). A review of dynamic vehicle routing problems. *European Journal of Operational Research*, 225(1), 1–11. <https://doi.org/10.1016/j.ejor.2012.08.015>
- [17] 许美贤, & 郑琰. (2021). 绿色车辆路径问题研究综述. 昆明理工大学学报 (自然科学版), 46(4), 158–170.
- [18] Azi, N., Gendreau, M., & Potvin, J.-Y. (2010). An exact algorithm for a vehicle routing problem with time windows and multiple use of vehicles. *European Journal of Operational Research*, 202(3), 756–763. <https://doi.org/10.1016/j.ejor.2009.06.034>

附录A 支撑材料

文件/文件夹名称	内容说明
main.py	项目核心主程序
scripts/generate_all_figures.py	生图代码
scripts/phase4_heuristic/heuristic.py	混合启发式调度算法求解模块
scripts/phase3_core/evaluator.py	时变路网计算与多车型路线综合成本评测
scripts/phase8_q3/dynamic_dispatcher.py	问题三中基于滚动时域的动态重调度代码
scripts/phase9_sensitivity/sensitivity_runner.py	敏感性分析与帕累托前沿分析
scripts/phase10_mc/stochastic_dispatcher.py	不确定性旅行时间下的蒙特卡洛鲁棒性仿真
data/	题目下发原始数据集
results/	计算结果

附录B 总控程序

```
1  # requirements.txt
2  ~~~ txt
3  numpy
4  pandas
5  scipy
6  matplotlib
7  seaborn
8  pulp
9  tqdm
10 ~~~
11 # data
12
13 # main.py
14 ~~~ python
15 import os
16 from pathlib import Path
17 import subprocess
18 import sys
```

```

19
20 def run_phase(script_path):
21     print(f"\n{'='*50}\nRunning {script_path}...\n{'='*50}")
22     result = subprocess.run([sys.executable, script_path], cwd=os.path.
        dirname(script_path))
23     if result.returncode != 0:
24         print(f"Error executing {script_path}")
25         sys.exit(1)
26
27 if __name__ == "__main__":
28     base_dir = os.path.abspath(os.path.dirname(__file__))
29     scripts_dir = base_dir / 'scripts'
30     phases = [
31         os.path.join(scripts_dir, 'phase1_ingest', 'ingest.py'),
32         os.path.join(scripts_dir, 'phase2_preprocess', 'preprocess.py'),
33         os.path.join(scripts_dir, 'phase3_core', 'evaluator.py'),
34         os.path.join(scripts_dir, 'phase4_heuristic', 'heuristic.py'),
35         os.path.join(scripts_dir, 'phase5_postopt', 'lp_solve.py')
36     ]
37
38     for phase in phases:
39         run_phase(phase)
40
41     print("\nAll phases executed successfully. Check the 'results'
        directory for outputs.")

```

附录C 基础配置代码

```

1 # constants.py
2 # 问题参数
3
4 # 时间段（以午夜为基准，单位为分钟）
5 BAN_START = 8 * 60 # 480
6 BAN_END = 16 * 60 # 960
7
8 # 费用与价格
9 STARTUP_COST = 400.0
10 WAIT_PENALTY = 1.0 / 3.0 # ¥/min
11 LATE_PENALTY = 5.0 / 6.0 # ¥/min
12
13 PRICE_FUEL = 7.61 # ¥/L
14 PRICE_ELEC = 1.64 # ¥/kWh
15

```

```

16 CO2_COST = 0.65 # ¥/kg
17 ETA_FUEL = 2.547 # kg/L
18 ETA_ELEC = 0.501 # kg/kWh
19
20 # 服务时间参数
21 S0 = 8.0 # minutes fixed
22 BETA_W = 0.004 # min/kg
23 BETA_V = 0.40 # min/m3
24
25 # 绿色区域半径
26 R_GREEN = 10.0 # km
27
28 # 车辆类型
29 VEHICLE_SPECS = {
30     'F-3000': {'type': 'fuel', 'Q_w': 3000.0, 'Q_v': 13.5, 'count': 60, '
31         alpha': 0.40},
32     'F-1500': {'type': 'fuel', 'Q_w': 1500.0, 'Q_v': 10.8, 'count': 50, '
33         alpha': 0.40},
34     'F-1250': {'type': 'fuel', 'Q_w': 1250.0, 'Q_v': 6.5, 'count': 50, '
35         alpha': 0.40},
36     'E-3000': {'type': 'electric', 'Q_w': 3000.0, 'Q_v': 15.0, 'count': 10,
37         'alpha': 0.35},
38     'E-1250': {'type': 'electric', 'Q_w': 1250.0, 'Q_v': 8.5, 'count': 15,
39         'alpha': 0.35},
40 }
41
42 # 速度参数 (mu_v, sigma_v) km/h
43 SPEED_PARAMS = {
44     'smooth': (55.3, 0.12),
45     'normal': (35.4, 5.22),
46     'congested': (9.8, 4.72)
47 }
48
49 # 时间段定义 (start_min, end_min, regime_name)
50 # 按开始时间排序
51 REGIMES = [
52     (0, 8*60, 'smooth'), # 在上午8点前默认采用平滑处理
53     (8*60, 9*60, 'congested'),
54     (9*60, 10*60, 'smooth'),
55     (10*60, 11*60+30, 'normal'),
56     (11*60+30, 13*60, 'congested'),
57     (13*60, 15*60, 'smooth'),
58     (15*60, 17*60, 'normal'),
59     (17*60, 19*60, 'congested'),
60 ]

```

```

1
2 @dataclass
3 class Customer:
4     id: int
5     x: float
6     y: float
7     demand_w: float
8     demand_v: float
9     tw_start: float
10    tw_end: float
11    is_green: bool = False
12
13 @dataclass
14 class Task:
15     # 客户的分拆包裹
16     task_id: int
17     customer_id: int
18     w: float
19     v: float
20     tw_start: float
21     tw_end: float
22     is_green: bool
23     service_time: float
24
25 @dataclass
26 class Vehicle:
27     vtype: str
28     is_electric: bool
29     Q_w: float
30     Q_v: float
31     alpha: float
32
33 @dataclass
34 class Route:
35     vtype: str
36     tasks: List[Task] = field(default_factory=list)
37     # 变量 z_w, z_v 可能是与任务配对的列表
38     splits_w: List[float] = field(default_factory=list)
39     splits_v: List[float] = field(default_factory=list)
40
41 @dataclass
42 class Subsequence:
43     D: float # 总时长

```

```

44     E: float # 最早可开始时间
45     L: float # 最早可开始时间
46     W: float # 等待成本
47     P: float # 后期成本

```

附录D 数据读入代码

```

1  import sys
2  import os
3  from pathlib import Path
4  import pandas as pd
5  import numpy as np
6  import matplotlib.pyplot as plt
7
8  # 将根目录添加到路径中以导入 phase0_setup
9  sys.path.append(str(Path(__file__).resolve().parent.parent))
10 from phase0_setup.constants import R_GREEN
11 from phase0_setup.datatypes import Customer
12
13 def parse_time(time_str):
14     if pd.isna(time_str):
15         return 0
16     parts = time_str.split(':')
17     return int(parts[0]) * 60 + int(parts[1])
18
19 def ingest_data(data_dir):
20     # 1. 加载 Customer contact information CSV 文件
21     coord_file = data_dir / 'Customer contact information.csv'
22     df_coords = pd.read_csv(coord_file)
23     coords = {}
24     for _, row in df_coords.iterrows():
25         coords[int(row['ID'])] = (float(row['X (km)']), float(row['Y (km)']
26                                     ))
27
28     # 2. 加载 distance matrix CSV 文件
29     dist_file = data_dir / 'Distance matrix.csv'
30     df_dist = pd.read_csv(dist_file, index_col=0)
31     D = df_dist.values
32
33     # 3. 加载 Order Informatio CSV 文件
34     order_file = data_dir / 'Order Information.csv'
35     df_orders = pd.read_csv(order_file)
36     # weight列标题中包含一个换行符: 'Weight\n'

```

```

36 weight_col = [c for c in df_orders.columns if 'Weight' in c][0]
37 df_orders[weight_col] = pd.to_numeric(df_orders[weight_col], errors='
coerce').fillna(0.0)
38 df_orders['Volume'] = pd.to_numeric(df_orders['Volume'], errors='coerce
').fillna(0.0)
39
40 orders_agg = df_orders.groupby('Target Customer ID').agg({
41     weight_col: 'sum',
42     'Volume': 'sum'
43 }).reset_index()
44 orders_agg.columns = ['customer_id', 'total_weight', 'total_volume']
45
46 # 4. 加载 Time Window CSV 文件
47 tw_file = data_dir / 'Time Window.csv'
48 df_tw = pd.read_csv(tw_file)
49 df_tw['tw_start'] = df_tw['Start time'].apply(parse_time)
50 df_tw['tw_end'] = df_tw['End Time'].apply(parse_time)
51
52 tw = {}
53 for _, row in df_tw.iterrows():
54     tw[int(row['Customer ID'])] = (row['tw_start'], row['tw_end'])
55
56 # 5. 合并统一
57 customers = {}
58 # DC is ID 0
59 customers[0] = Customer(
60     id=0, x=coords[0][0], y=coords[0][1],
61     demand_w=0.0, demand_v=0.0,
62     tw_start=0.0, tw_end=24*60.0
63 )
64
65 for i in range(1, 99):
66     x, y = coords[i]
67     # 获取需求
68     demand_w = 0.0
69     demand_v = 0.0
70     order_match = orders_agg[orders_agg['customer_id'] == i]
71     if not order_match.empty:
72         demand_w = order_match['total_weight'].values[0]
73         demand_v = order_match['total_volume'].values[0]
74
75     # 获取时区 (若未指定, 则使用 [0, 24*60])
76     tw_start, tw_end = tw.get(i, (0.0, 24*60.0))
77
78     customers[i] = Customer(

```

```

79         id=i, x=x, y=y,
80         demand_w=demand_w, demand_v=demand_v,
81         tw_start=tw_start, tw_end=tw_end
82     )
83
84     return coords, D, df_orders, tw, customers
85
86 def plot_layout(customers, figures_dir):
87     plt.figure(figsize=(10, 10))
88
89     x_green = []
90     y_green = []
91     x_reg = []
92     y_reg = []
93
94     for i in range(1, 99):
95         c = customers[i]
96         # 在phase1, 绿色区域的逻辑并未严格执行, 但为了便于说明, 在此对其进行着色
97         dist_to_center = np.sqrt(c.x**2 + c.y**2)
98         if dist_to_center <= R_GREEN:
99             x_green.append(c.x)
100            y_green.append(c.y)
101         else:
102             x_reg.append(c.x)
103             y_reg.append(c.y)
104
105     plt.scatter(x_reg, y_reg, c='blue', label='Regular Customer', alpha
=0.6)
106     plt.scatter(x_green, y_green, c='green', label='Green Zone Customer',
alpha=0.8)
107
108     # DC
109     dc = customers[0]
110     plt.scatter(dc.x, dc.y, c='red', marker='*', s=200, label='DC')
111
112     # 绿色区域圆圈
113     circle = plt.Circle((0, 0), R_GREEN, color='green', fill=False,
linestyle='--', label='Green Zone Boundary')
114     plt.gca().add_patch(circle)
115
116     plt.xlim(-40, 40)
117     plt.ylim(-40, 40)
118     plt.axhline(0, color='black', linewidth=0.5, alpha=0.5)
119     plt.axvline(0, color='black', linewidth=0.5, alpha=0.5)

```

```

120     plt.grid(True, alpha=0.3)
121     plt.legend()
122     plt.title('Figure F1: Customer Layout')
123     plt.xlabel('X (km)')
124     plt.ylabel('Y (km)')
125
126     plt.savefig(os.path.join(figures_dir, 'f01_customer_layout.png'), dpi
127 =300, bbox_inches='tight')
128     plt.close()
129
130 def audit_data(D, customers, results_dir):
131     audit_path = results_dir / 'data_audit.txt'
132     with open(audit_path, 'w') as f:
133         f.write("--- Data Quality Audit ---\n")
134         f.write(f"Total customers (excluding DC): {len(customers)-1}\n")
135
136         # 需求为零的客户
137         zero_demand = [c.id for c in customers.values() if c.id != 0 and c.
138 demand_w == 0 and c.demand_v == 0]
139         f.write(f"Customers with zero demand: {len(zero_demand)} (IDs: {
140 zero_demand})\n")
141
142         # 距离矩阵校验
143         n = D.shape[0]
144         is_sym = np.allclose(D, D.T, atol=1e-5)
145         f.write(f"Distance matrix symmetric: {is_sym}\n")
146
147         green_count = sum(1 for c in customers.values() if c.id != 0 and np
148 .sqrt(c.x**2 + c.y**2) <= R_GREEN)
149         f.write(f"Green zone customers: {green_count}\n")
150
151 if __name__ == "__main__":
152     base_dir = Path(__file__).resolve().parent.parent.parent
153     data_dir = base_dir / 'data'
154     results_dir = base_dir / 'results'
155     figures_dir = base_dir / 'figures'
156     coords, D, df_orders, tw, customers = ingest_data(data_dir)
157
158     plot_layout(customers, figures_dir)
159     audit_data(D, customers, results_dir)
160     print("Phase 1 executed successfully.")

```

附录E 预处理代码

```
1 import sys
2 import os
3 from pathlib import Path
4 import json
5 import numpy as np
6 import matplotlib.pyplot as plt
7
8 sys.path.append(str(Path(__file__).resolve().parent.parent))
9 from phase0_setup.constants import SPEED_PARAMS, REGIMES, R_GREEN,
    VEHICLE_SPECS, SO, BETA_W, BETA_V
10 from phase0_setup.datatypes import Task
11 from phase1_ingest.ingest import ingest_data
12
13 def calibrate_pace():
14     pace_table = {}
15     for name, (mu_v, sigma_v) in SPEED_PARAMS.items():
16         mu_p = 60.0 / mu_v
17         sigma_p = (60.0 * sigma_v) / (mu_v ** 2)
18         pace_table[name] = {'mu_p': mu_p, 'sigma_p': sigma_p}
19     return pace_table
20
21 def get_regime_at_time(t):
22     t_mod = t % (24 * 60)
23     for start, end, name in REGIMES:
24         if start <= t_mod < end:
25             return name
26     return 'smooth' # 默认备选方案
27
28 def build_travel_time_lookup(D, pace_table):
29     n = D.shape[0]
30     T_lookup = np.zeros((n, n, 180))
31     time_grid = np.arange(420, 420 + 180 * 5, 5) # 07:00 to 21:55
32     for i in range(n):
33         for j in range(n):
34             if i == j:
35                 continue
36             d_ij = D[i, j]
37             for t_idx, t0 in enumerate(time_grid):
38                 d_rem = d_ij
39                 t_curr = t0
40                 while d_rem > 1e-6:
41                     t_mod = t_curr % (24 * 60)
```

```

42         # 查找当前工作区边界
43         r_name = 'smooth'
44         r_end = 24 * 60
45         for start, end, name in REGIMES:
46             if start <= t_mod < end:
47                 r_name = name
48                 r_end = t_curr - t_mod + end # 恢复为绝对路径
49                 break
50         mu_p = pace_table[r_name]['mu_p']
51         time_avail = r_end - t_curr
52         dist_avail = time_avail / mu_p
53
54         if d_rem <= dist_avail:
55             t_curr += d_rem * mu_p
56             d_rem = 0
57         else:
58             t_curr = r_end
59             d_rem -= dist_avail
60
61         T_lookup[i, j, t_idx] = t_curr - t0
62
63     return T_lookup, time_grid
64
65 def compute_G_ij(coords):
66     n = len(coords)
67     G = np.zeros((n, n), dtype=bool)
68     for i in range(n):
69         P_i = np.array(coords[i])
70         for j in range(n):
71             if i == j:
72                 continue
73             P_j = np.array(coords[j])
74
75             dP = P_j - P_i
76             A = np.dot(dP, dP)
77             B = 2 * np.dot(P_i, dP)
78             C = np.dot(P_i, P_i)
79
80             if A == 0:
81                 continue
82             t_star = -B / (2 * A)
83
84             if 0 <= t_star <= 1 and (C - B**2 / (4 * A)) < R_GREEN**2:
85                 G[i, j] = True
86             elif t_star < 0 and C < R_GREEN**2:

```

```

87         G[i, j] = True
88     elif t_star > 1 and (A + B + C) < R_GREEN**2:
89         G[i, j] = True
90         return G
91
92 def compute_detour(D, coords, G):
93     n = len(coords)
94     detour = np.copy(D)
95     for i in range(n):
96         P_i = np.array(coords[i])
97         D_i = np.linalg.norm(P_i)
98         for j in range(n):
99             if not G[i, j]:
100                 continue
101                 P_j = np.array(coords[j])
102                 D_j = np.linalg.norm(P_j)
103
104                 if D_i <= R_GREEN or D_j <= R_GREEN:
105                     detour[i, j] = 1e9 # 不可行
106                     continue
107                     l_i = np.sqrt(D_i**2 - R_GREEN**2)
108                     l_j = np.sqrt(D_j**2 - R_GREEN**2)
109
110                     dot_val = np.dot(P_i, P_j) / (D_i * D_j)
111                     dot_val = np.clip(dot_val, -1.0, 1.0)
112                     delta_theta = np.arccos(dot_val)
113
114                     gamma_i = np.arccos(R_GREEN / D_i)
115                     gamma_j = np.arccos(R_GREEN / D_j)
116
117                     theta_arc = delta_theta - gamma_i - gamma_j
118                     if theta_arc > 0:
119                         detour[i, j] = l_i + l_j + R_GREEN * theta_arc
120                     else:
121                         detour[i, j] = D[i, j]
122
123     return detour
124
125 def discretise_packets(customers):
126     packets = []
127     Q_w_max = VEHICLE_SPECS['F-3000']['Q_w']
128     Q_v_max = VEHICLE_SPECS['F-3000']['Q_v']
129
130     task_id = 0
131     for i in range(1, 99):

```

```

132     c = customers[i]
133     if c.demand_w == 0 and c.demand_v == 0:
134         continue
135         n_w = int(np.ceil(c.demand_w / Q_w_max))
136     n_v = int(np.ceil(c.demand_v / Q_v_max))
137     n_i = max(1, n_w, n_v)
138
139     w_split = c.demand_w / n_i
140     v_split = c.demand_v / n_i
141     svc_time = S0 + BETA_W * w_split + BETA_V * v_split
142
143     c.is_green = bool(np.linalg.norm(np.array([c.x, c.y])) <= R_GREEN)
144
145     for _ in range(n_i):
146         packets.append(Task(
147             task_id=task_id,
148             customer_id=c.id,
149             w=w_split,
150             v=v_split,
151             tw_start=c.tw_start,
152             tw_end=c.tw_end,
153             is_green=c.is_green,
154             service_time=svc_time
155         ))
156         task_id += 1
157     return packets
158
159 def plot_demand(customers, packets, figures_dir):
160     cust_w = [c.demand_w for c in customers.values() if c.id != 0 and c.
161 demand_w > 0]
162     pkt_w = [p.w for p in packets]
163
164     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
165     ax1.hist(cust_w, bins=20, color='skyblue', edgecolor='black')
166     ax1.axvline(3000, color='red', linestyle='dashed', linewidth=2, label='
167 3000 kg Limit')
168     ax1.set_title('Customer Demand (Weight)')
169     ax1.legend()
170
171     ax2.hist(pkt_w, bins=20, color='lightgreen', edgecolor='black')
172     ax2.axvline(3000, color='red', linestyle='dashed', linewidth=2)
173     ax2.set_title('Packet Demand (Weight) after Splitting')
174
175     plt.suptitle('Figure F4: Demand Distribution')
176     plt.tight_layout()

```

```

175     plt.savefig(os.path.join(figures_dir, 'f04_demand_distribution.png'),
176                 dpi=300)
177     plt.close()
178
179 def plot_pace(pace_table, figures_dir):
180     times = np.arange(420, 1320)
181     mu_vals = []
182     sigma_vals = []
183     for t in times:
184         r = get_regime_at_time(t)
185         mu_vals.append(pace_table[r]['mu_p'])
186         sigma_vals.append(pace_table[r]['sigma_p'])
187
188     mu_vals = np.array(mu_vals)
189     sigma_vals = np.array(sigma_vals)
190
191     plt.figure(figsize=(10, 5))
192     plt.plot(times / 60, mu_vals, label='Mean Pace (min/km)', color='blue')
193     plt.fill_between(times / 60, mu_vals - sigma_vals, mu_vals + sigma_vals,
194                     color='blue', alpha=0.2, label='±1 Sigma')
195
196     plt.axvspan(8, 16, color='red', alpha=0.1, label='Ban Window')
197
198     plt.xlabel('Hour of Day')
199     plt.ylabel('Pace (min/km)')
200     plt.title('Figure F2: Time-Dependent Pace Profile')
201     plt.legend()
202     plt.grid(True)
203     plt.savefig(os.path.join(figures_dir, 'f02_pace_profile.png'), dpi=300)
204     plt.close()
205
206 def plot_g_ij(G, figures_dir):
207     plt.figure(figsize=(8, 8))
208     plt.imshow(G, cmap='Blues', aspect='auto')
209     plt.title('Figure F3: Arc Penetration Matrix $G_{ij}$')
210     plt.colorbar(label='Penetrates Green Zone')
211     plt.savefig(os.path.join(figures_dir, 'f03_arc_penetration.png'), dpi=
212                 =300)
213     plt.close()
214
215 if __name__ == "__main__":
216     base_dir = Path(__file__).resolve().parent.parent.parent
217     data_dir = base_dir / 'data'
218     results_dir = base_dir / 'results'
219     precomp_dir = results_dir / 'precomputed'

```

```

217     figures_dir = base_dir / 'figures'
218     coords, D, df_orders, tw, customers = ingest_data(data_dir)
219
220     pace_table = calibrate_pace()
221     T_lookup, time_grid = build_travel_time_lookup(D, pace_table)
222
223     G = compute_G_ij(coords)
224     detour = compute_detour(D, coords, G)
225
226     packets = discretise_packets(customers)
227
228     precomp_dir.mkdir(parents=True, exist_ok=True)
229     np.save(precomp_dir / 'T_lookup.npy', T_lookup)
230     np.save(precomp_dir / 'G_ij.npy', G)
231     np.save(precomp_dir / 'detour.npy', detour)
232
233     with open(precomp_dir / 'packets.json', 'w') as f:
234         json.dump([vars(p) for p in packets], f, indent=2)
235
236     plot_demand(customers, packets, figures_dir)
237     plot_pace(pace_table, figures_dir)
238     plot_g_ij(G, figures_dir)
239
240     print(f"Phase 2 completed. Discretized into {len(packets)} packets.")

```

附录F 核心评估器代码

```

1  import numpy as np
2  import sys
3  import os
4  from pathlib import Path
5
6  sys.path.append(str(Path(__file__).resolve().parent.parent))
7  from phase0_setup.constants import VEHICLE_SPECS, REGIMES, WAIT_PENALTY,
   LATE_PENALTY, PRICE_ELEC, PRICE_FUEL, CO2_COST, ETA_ELEC, ETA_FUEL,
   SPEED_PARAMS, R_GREEN
8
9  class RouteEvaluator:
10     def __init__(self, T_lookup, G_ij, detour, time_grid, packets,
   customers, green_ban=False):
11         self.green_ban = green_ban
12         self.T = T_lookup
13         self.G = G_ij

```

```

14         self.detour = detour
15         self.time_grid = time_grid
16         self.packets = {p.task_id: p for p in packets}
17         self.customers = customers
18
19         # 敏感性分析中的动态覆盖
20         self.PRICE_FUEL = PRICE_FUEL
21         self.CO2_COST = CO2_COST
22         self.PRICE_ELEC = PRICE_ELEC
23
24         # 计算配速表
25         self.pace_table = {}
26         for name, (mu_v, sigma_v) in SPEED_PARAMS.items():
27             mu_p = 60.0 / mu_v
28             sigma_p = (60.0 * sigma_v) / (mu_v ** 2)
29             self.pace_table[name] = {'mu_p': mu_p, 'sigma_p': sigma_p}
30
31     def get_travel_time(self, i, j, t_dep, use_detour=False):
32         if i == j:
33             return 0.0
34         t_dep_clip = np.clip(t_dep, self.time_grid[0], self.time_grid
35 [-1])
36         idx = (t_dep_clip - self.time_grid[0]) / 5.0
37         idx0 = int(np.floor(idx))
38         idx1 = min(idx0 + 1, len(self.time_grid) - 1)
39         w = idx - idx0
40         tt = (1 - w) * self.T[i, j, idx0] + w * self.T[i, j, idx1]
41
42         if use_detour:
43             P_i = np.array([self.customers[i].x, self.customers[i].y])
44             P_j = np.array([self.customers[j].x, self.customers[j].y])
45             dist_straight = np.linalg.norm(P_i - P_j)
46             dist_ratio = self.detour[i, j] / max(1e-6, dist_straight)
47             tt *= dist_ratio
48
49         return tt
50
51     def base_invariants(self, service_time, tw_start, tw_end):
52         return (service_time, tw_start, tw_end, 0.0, 0.0)
53
54     def concat_invariants(self, inv1, inv2, travel_time):
55         D1, E1, L1, W1, P1 = inv1
56         D2, E2, L2, W2, P2 = inv2
57

```

```

58     D_new = D1 + travel_time + D2
59     E_new = max(E1, E2 - D1 - travel_time)
60     L_new = min(L1, L2 - D1 - travel_time)
61
62     W_new = W1 + W2 + WAIT_PENALTY * max(0, E2 - E1 - D1 - travel_time)
63     P_new = P1 + P2 + LATE_PENALTY * max(0, L1 + D1 + travel_time - L2)
64
65     return (D_new, E_new, L_new, W_new, P_new)
66
67     def evaluate_route(self, packet_ids, v_type, r_idx=-1):
68         nodes = [0] + [self.packets[pid].customer_id for pid in packet_ids]
69         + [0]
70         svc_times = [0.0] + [self.packets[pid].service_time for pid in
71 packet_ids] + [0.0]
72         tw_starts = [0.0] + [self.packets[pid].tw_start for pid in
73 packet_ids] + [0.0]
74         tw_ends = [24*60.0] + [self.packets[pid].tw_end for pid in
75 packet_ids] + [24*60.0]
76
77         total_w = sum(self.packets[pid].w for pid in packet_ids)
78         total_v = sum(self.packets[pid].v for pid in packet_ids)
79         spec = VEHICLE_SPECS[v_type]
80
81         if total_w > spec['Q_w'] + 1e-6 or total_v > spec['Q_v'] + 1e-6:
82             return float('inf'), False, {"reason": "Capacity"}
83
84         is_ev = (spec['type'] == 'electric')
85         fixed_cost = 400.0
86         alpha_star = spec['alpha']
87
88         best_cost = float('inf')
89         best_details = {}
90
91         # 对从DC出发的可能时间进行网格搜索
92         # 由于等待惩罚为 1/3, 而迟到惩罚为 5/6, 我们应尽可能晚出发以避免等
93 待,
94         # 但也要足够早出发以避免迟到惩罚。我们可以直接测试从 420 到 1200 的
95 出发时间, 步长为 30。
96         # 实际上, 我们只需检查与每个运输周 (TW) 开始时间对齐的出发时间。
97         candidate_deps = [420.0]
98         for k in range(1, len(nodes)):
99             # 粗略估算 tw_starts 的反向传播
100             candidate_deps.append(max(420.0, tw_starts[k] - 120.0))
101             candidate_deps.append(max(420.0, tw_starts[k] - 60.0))
102             candidate_deps.append(max(420.0, tw_starts[k]))

```

```

97
98     for dep_DC in sorted(list(set(candidate_deps))):
99         route_dist = 0.0
100         energy_cost = 0.0
101         soft_penalty = 0.0
102         current_w = total_w
103
104         t_curr = dep_DC
105         feasible = True
106         for k in range(1, len(nodes)):
107             i = nodes[k-1]
108             j = nodes[k]
109
110             use_detour = False
111             if self.green_ban and not is_ev and self.G[i, j]:
112                 tt_est = self.get_travel_time(i, j, t_curr, False)
113                 t_arr_est = t_curr + tt_est
114                 if 480 <= t_arr_est <= 960 or 480 <= t_curr <= 960:
115                     if self.detour[i, j] >= 1e8:
116                         t_curr = max(t_curr, 960.0)
117                     else:
118                         use_detour = True
119                 P_i = np.array([self.customers[i].x, self.customers[i].y])
120                 P_j = np.array([self.customers[j].x, self.customers[j].y])
121                 dist_straight = np.linalg.norm(P_i - P_j)
122                 d_ij = self.detour[i, j] if use_detour else dist_straight
123                 route_dist += d_ij
124
125                 tt = self.get_travel_time(i, j, t_curr, use_detour)
126                 t_arr = t_curr + tt
127
128                 if tt > 1e-6 and d_ij > 1e-6:
129                     v_kmh = d_ij / (tt / 60.0)
130                     rho = current_w / spec['Q_w']
131
132                     if is_ev:
133                         epk = 0.0014 * v_kmh**2 - 0.12 * v_kmh + 36.19
134                         rate_per_km = (epk / 100.0) * (1 + spec['alpha'] *
rho)
135
136                         energy_cost += rate_per_km * d_ij * self.PRICE_ELEC
137                     else:
138                         fpk = 0.0025 * v_kmh**2 - 0.2554 * v_kmh + 31.75
139                         rate_per_km = (fpk / 100.0) * (1 + spec['alpha'] *
rho)
140
141                         fuel_c = rate_per_km * d_ij * self.PRICE_FUEL

```

```

140         carbon_c = rate_per_km * d_ij * ETA_FUEL * self.
CO2_COST
141         energy_cost += fuel_c + carbon_c
142
143         if k < len(nodes) - 1:
144             pid = packet_ids[k-1]
145             current_w -= self.packets[pid].w
146
147             # Penalty at node k
148             if t_arr < tw_starts[k]:
149                 soft_penalty += WAIT_PENALTY * (tw_starts[k] - t_arr)
150                 t_curr = tw_starts[k] + svc_times[k]
151             elif t_arr > tw_ends[k]:
152                 soft_penalty += LATE_PENALTY * (t_arr - tw_ends[k])
153                 t_curr = t_arr + svc_times[k]
154             else:
155                 t_curr = t_arr + svc_times[k]
156
157         total_cost = fixed_cost + energy_cost + soft_penalty
158         if total_cost < best_cost:
159             best_cost = total_cost
160             best_details = {
161                 "fixed_cost": fixed_cost,
162                 "energy_cost": energy_cost,
163                 "penalty": soft_penalty,
164                 "dist": route_dist,
165                 "optimal_dep": dep_DC
166             }
167
168         return best_cost, True, best_details
169
170 if __name__ == "__main__":
171     # 简单的测试逻辑
172     base_dir = Path(__file__).resolve().parent.parent.parent
173     data_dir = base_dir / 'data'
174     precomp_dir = base_dir / 'results' / 'precomputed'
175     from phase1_ingest.ingest import ingest_data
176     coords, D, df_orders, tw, customers = ingest_data(data_dir)
177
178     import json
179     T_lookup = np.load(precomp_dir / 'T_lookup.npy')
180     G_ij = np.load(precomp_dir / 'G_ij.npy')
181     detour = np.load(precomp_dir / 'detour.npy')
182
183     from phase0_setup.datatypes import Task

```

```

184     with open(precomp_dir / 'packets.json', 'r') as f:
185         packet_dicts = json.load(f)
186         packets = [Task(**d) for d in packet_dicts]
187
188         time_grid = np.arange(420, 420 + 180 * 5, 5)
189
190         evaluator = RouteEvaluator(T_lookup, G_ij, detour, time_grid, packets,
191                                   customers)
192
193         # 使用单包裹路线进行测试
194         if len(packets) > 0:
195             cost, feas, det = evaluator.evaluate_route([packets[0].task_id], 'F
-3000')
196             print(f"Test Route F-3000 Cost: {cost:.2f}, Feasible: {feas},
197                   Details: {det}")
198
199             cost, feas, det = evaluator.evaluate_route([packets[0].task_id], 'E
-3000')
200             print(f"Test Route E-3000 Cost: {cost:.2f}, Feasible: {feas},
201                   Details: {det}")
202             print("Phase 3 executed successfully.")

```

附录G 启发式求解代码

```

1  import sys
2  import os
3  from pathlib import Path
4  import json
5  import numpy as np
6  from copy import deepcopy
7  import time
8
9  sys.path.append(str(Path(__file__).resolve().parent.parent))
10 from phase0_setup.constants import VEHICLE_SPECS
11 from phase0_setup.datatypes import Task
12 from phase1_ingest.ingest import ingest_data
13 from phase3_core.evaluator import RouteEvaluator
14
15 class Solution:
16     def __init__(self):
17         self.routes = []
18
19     def total_cost(self):

```

```

20         return sum(r['cost'] for r in self.routes)
21
22 def greedy_insertion(evaluator, packets, green_ban=False, initial_routes=
None, v_counts_override=None):
23     sol = Solution()
24     if initial_routes is not None:
25         sol.routes = deepcopy(initial_routes)
26
27     # 按绿色区域 (True 优先) 排序数据包, 然后按 tw_start 升序, 最后按权重降
序
28     sorted_packets = sorted(packets, key=lambda p: (not p.is_green, p.
tw_start, -p.w))
29
30     # 可用车辆行程。制造E-3000短缺, 迫使燃油车进入绿色区域。
31     if v_counts_override is not None:
32         v_counts = deepcopy(v_counts_override)
33     else:
34         v_counts = {
35             'E-3000': 10, # 严格限制 “仅存10台E-3000”
36             'F-3000': 120, # 允许重复使用行程以满足总需求
37             'F-1500': 50,
38             'E-1250': 15,
39             'F-1250': 50
40         }
41
42     theta = 5000.0 # 开通新路线的门槛
43     for p in sorted_packets:
44         best_cost_delta = float('inf')
45         best_r_idx = -1
46         best_p_idx = -1
47         best_new_route_cost = float('inf')
48         best_new_v_type = None
49         allowed_vtypes = list(v_counts.keys())
50         # 我们不再严格强制要求使用电动汽车。燃油车可以服务于绿色客户, 但将
受到处罚
51         # 尝试插入到现有路线上
52         for r_idx, r in enumerate(sol.routes):
53             if r['v_type'] not in allowed_vtypes:
54                 continue
55             old_cost = r['cost']
56             # 尝试所有位置
57             for p_idx in range(len(r['packets']) + 1):
58                 new_packets = list(r['packets'])
59                 new_packets.insert(p_idx, p.task_id)
60

```

```

61         cost, feas, det = evaluator.evaluate_route(new_packets, r['
v_type'], r_idx=r_idx)
62         if feas:
63             delta = cost - old_cost
64             if delta < best_cost_delta:
65                 best_cost_delta = delta
66                 best_r_idx = r_idx
67                 best_p_idx = p_idx
68
69         # 试着开通一条新路线
70         for v_type in allowed_vtypes:
71             count = v_counts.get(v_type, 0)
72             if count > 0:
73                 cost, feas, det = evaluator.evaluate_route([p.task_id],
v_type)
74                 if feas and cost < best_new_route_cost:
75                     best_new_route_cost = cost
76                     best_new_v_type = v_type
77
78             if best_cost_delta < theta and best_r_idx != -1:
79                 # 插入到现有路线
80                 r = sol.routes[best_r_idx]
81                 r['packets'].insert(best_p_idx, p.task_id)
82                 r['cost'], feas, det = evaluator.evaluate_route(r['packets'], r
['v_type'], r_idx=best_r_idx)
83                 r['details'] = det
84             else:
85                 # 开通一条新路线
86                 if best_new_v_type is None:
87                     # 排查失败原因
88                     for v_type, count in v_counts.items():
89                         if count > 0:
90                             cost, feas, det = evaluator.evaluate_route([p.
task_id], v_type)
91                             print(f"v_type {v_type}: feas={feas}, det={det}")
92                             raise ValueError(f"Cannot schedule packet {p.task_id}")
93                             v_counts[best_new_v_type] -= 1
94                             cost, feas, det = evaluator.evaluate_route([p.task_id],
best_new_v_type)
95                             sol.routes.append({
96                                 'v_type': best_new_v_type,
97                                 'packets': [p.task_id],
98                                 'cost': cost,
99                                 'details': det
100                             })

```

```

101
102     return sol, v_counts
103
104 def local_search(evaluator, sol, max_iters=10, green_ban=False, event_loc=
None):
105     improved = True
106     iters = 0
107     while improved and iters < max_iters:
108         improved = False
109         iters += 1
110         print(f"LS Iteration {iters}, Current Cost: {sol.total_cost():.2f}"
)
111
112         # 1. 迁移
113         for r1_idx, r1 in enumerate(sol.routes):
114             for i in range(len(r1['packets'])):
115                 pid = r1['packets'][i]
116
117                 if event_loc is not None:
118                     c_id = evaluator.packets[pid].customer_id
119                     cx, cy = evaluator.customers[c_id].x, evaluator.
customers[c_id].y
120                     dist_to_event = np.sqrt((cx - event_loc[0])**2 + (cy -
event_loc[1])**2)
121                     if dist_to_event > 5.0:
122                         continue
123                     for r2_idx, r2 in enumerate(sol.routes):
124                         if r1_idx == r2_idx:
125                             continue
126                         for j in range(len(r2['packets']) + 1):
127                             new_r1 = list(r1['packets'])
128                             new_r1.pop(i)
129                             new_r2 = list(r2['packets'])
130                             new_r2.insert(j, pid)
131
132                             c1, f1, d1 = evaluator.evaluate_route(new_r1, r1['
v_type'], r_idx=r1_idx) if len(new_r1) > 0 else (0.0, True, {})
133                             c2, f2, d2 = evaluator.evaluate_route(new_r2, r2['
v_type'], r_idx=r2_idx)
134
135                             if f1 and f2:
136                                 old_cost = r1['cost'] + r2['cost']
137                                 new_cost = c1 + c2
138                                 if new_cost < old_cost - 1e-4:
139                                     r1['cost'] = c1

```

```

140         r1['details'] = d1
141         r2['packets'] = new_r2
142         r2['cost'] = c2
143         r2['details'] = d2
144         improved = True
145         break if improved: break
146     if improved: break
147     if improved: break
148     #清理空路线
149     sol.routes = [r for r in sol.routes if len(r['packets']) > 0]
150
151     return sol
152
153 if __name__ == "__main__":
154     base_dir = Path(__file__).resolve().parent.parent.parent
155     data_dir = base_dir / 'data'
156     precomp_dir = base_dir / 'results' / 'precomputed'
157     results_dir = base_dir / 'results'
158     coords, D, df_orders, tw, customers = ingest_data(data_dir)
159
160     T_lookup = np.load(precomp_dir / 'T_lookup.npy')
161     G_ij = np.load(precomp_dir / 'G_ij.npy')
162     detour = np.load(precomp_dir / 'detour.npy')
163
164     with open(precomp_dir / 'packets.json', 'r') as f:
165         packet_dicts = json.load(f)
166         packets = [Task(**d) for d in packet_dicts]
167
168     time_grid = np.arange(420, 420 + 180 * 5, 5)
169
170     evaluator = RouteEvaluator(T_lookup, G_ij, detour, time_grid, packets,
171                                customers)
172
173     print("Starting construction heuristic...")
174     t0 = time.time()
175     # 对于第4阶段, 我们假设Q1 (无绿色禁令)
176     green_ban = False
177     sol, v_counts = greedy_insertion(evaluator, packets, green_ban=
178     green_ban)
179     t1 = time.time()
180     print(f"Construction finished in {t1-t0:.2f}s. Total cost: {sol.
181     total_cost():.2f}")
182
183     print("Starting local search...")
184     sol = local_search(evaluator, sol, green_ban=green_ban)

```

```

182     t2 = time.time()
183     print(f"Local search finished in {t2-t1:.2f}s. Final cost: {sol.
total_cost():.2f}")
184     print(f"Total routes opened: {len(sol.routes)}")
185
186     with open(results_dir / 'phaseA_solution.json', 'w') as f:
187         json.dump(sol.routes, f, indent=2)
188
189     print("Phase 4 executed successfully.")

```

附录H 后优化代码

```

1  import sys
2  import os
3  from pathlib import Path
4  import json
5
6  sys.path.append(str(Path(__file__).resolve().parent.parent))
7  from phase0_setup.datatypes import Task
8
9  def consolidate_packets(sol_routes, packets):
10     #将同一车辆上同一客户的连续货包进行合并。这符合离散分配的要求，并代表了
    简化的B阶段优化后处理（合并拆分）。
11     packet_dict = {p.task_id: p for p in packets}
12     final_routes = []
13
14     for r in sol_routes:
15         if not r['packets']:
16             continue
17         merged_customers = []
18         current_cust = None
19         current_w = 0.0
20         current_v = 0.0
21         for pid in r['packets']:
22             p = packet_dict[pid]
23             if current_cust is None:
24                 current_cust = p.customer_id
25                 current_w = p.w
26                 current_v = p.v
27             elif p.customer_id == current_cust:
28                 current_w += p.w
29                 current_v += p.v
30             else:

```

```

31         merged_customers.append({
32             'customer_id': current_cust,
33             'delivered_w': current_w,
34             'delivered_v': current_v
35         })
36         current_cust = p.customer_id
37         current_w = p.w
38         current_v = p.v
39
40     if current_cust is not None:
41         merged_customers.append({
42             'customer_id': current_cust,
43             'delivered_w': current_w,
44             'delivered_v': current_v
45         })
46
47     final_routes.append({
48         'v_type': r['v_type'],
49         'visits': merged_customers,
50         'cost': r['cost'],
51         'details': r.get('details', {})
52     })
53
54     return final_routes
55
56 if __name__ == "__main__":
57     base_dir = Path(__file__).resolve().parent.parent.parent
58     results_dir = base_dir / 'results'
59     precomp_dir = results_dir / 'precomputed'
60     with open(precomp_dir / 'packets.json', 'r') as f:
61         packet_dicts = json.load(f)
62         packets = [Task(**d) for d in packet_dicts]
63
64     sol_path = results_dir / 'phaseA_solution.json'
65     if os.path.exists(sol_path):
66         with open(sol_path, 'r') as f:
67             sol_routes = json.load(f)
68
69     final_routes = consolidate_packets(sol_routes, packets)
70
71     with open(results_dir / 'phaseB_solution.json', 'w') as f:
72         json.dump(final_routes, f, indent=2)
73
74     print(f"Phase 5 (Post-Optimisation/Consolidation) completed. Total
consolidated routes: {len(final_routes)}")

```

```

75     else:
76         print("No Phase A solution found.")

```

附录I 问题一代码

```

1  import sys
2  import os
3  from pathlib import Path
4  import json
5  import pandas as pd
6  import numpy as np
7  import matplotlib.pyplot as plt
8  import matplotlib.colors as mcolors
9
10 sys.path.append(str(Path(__file__).resolve().parent.parent))
11 from phase0_setup.constants import VEHICLE_SPECS, R_GREEN
12 from phase0_setup.datatypes import Task
13 from phase1_ingest.ingest import ingest_data
14
15 def generate_summary(routes):
16     by_vtype = {}
17     for r in routes:
18         vt = r['v_type']
19         if vt not in by_vtype:
20             by_vtype[vt] = 0
21         by_vtype[vt] += 1
22         physical_startup = 0
23     for vt, trips in by_vtype.items():
24         physical_used = min(trips, VEHICLE_SPECS[vt]['count'])
25         physical_startup += physical_used * 400.0
26         summary = {
27             'n_routes': len(routes),
28             'total_km': sum(r['details']['dist'] for r in routes),
29             'startup_cost': physical_startup,
30             'energy_cost': sum(r['details']['energy_cost'] for r in routes),
31             'penalty_cost': sum(r['details']['penalty'] for r in routes),
32             'by_vtype': by_vtype
33         }
34
35     summary['total_cost'] = summary['startup_cost'] + summary['energy_cost']
36     + summary['penalty_cost']
37
38     return summary

```

```

38
39 def generate_csvs(routes, q1_dir):
40     stops_data = []
41     routes_data = []
42
43     for r_idx, r in enumerate(routes):
44         vt = r['v_type']
45         details = r['details']
46
47         # 简化重构
48         stops_data.append({
49             'route_id': r_idx,
50             'vehicle_type': vt,
51             'stop_order': 0,
52             'customer_id': 0, # 从DC出发
53             'delivered_w': 0,
54             'delivered_v': 0
55         })
56
57         tot_w = 0
58         tot_v = 0
59         for s_idx, visit in enumerate(r['visits']):
60             stops_data.append({
61                 'route_id': r_idx,
62                 'vehicle_type': vt,
63                 'stop_order': s_idx + 1,
64                 'customer_id': visit['customer_id'],
65                 'delivered_w': visit['delivered_w'],
66                 'delivered_v': visit['delivered_v']
67             })
68             tot_w += visit['delivered_w']
69             tot_v += visit['delivered_v']
70
71         stops_data.append({
72             'route_id': r_idx,
73             'vehicle_type': vt,
74             'stop_order': len(r['visits']) + 1,
75             'customer_id': 0, # DC 回归
76             'delivered_w': 0,
77             'delivered_v': 0
78         })
79
80         routes_data.append({
81             'route_id': r_idx,
82             'vehicle_type': vt,

```

```

83         'n_stops': len(r['visits']),
84         'total_load_w': tot_w,
85         'total_load_v': tot_v,
86         'distance_km': details['dist'],
87         'startup_cost': details['fixed_cost'],
88         'energy_cost': details['energy_cost'],
89         'penalty_cost': details['penalty'],
90         'total_cost': r['cost'],
91         'depart_dc': details['optimal_dep']
92     })
93
94     pd.DataFrame(stops_data).to_csv(q1_dir / 'stops.csv', index=False)
95     pd.DataFrame(routes_data).to_csv(q1_dir / 'routes_summary.csv', index=
False)
96
97 def plot_routes(routes, customers, figures_dir):
98     plt.figure(figsize=(12, 12))
99
100     # 绘制客户图
101     x_green, y_green = [], []
102     x_reg, y_reg = [], []
103     for i in range(1, 99):
104         c = customers[i]
105         if np.sqrt(c.x**2 + c.y**2) <= R_GREEN:
106             x_green.append(c.x)
107             y_green.append(c.y)
108         else:
109             x_reg.append(c.x)
110             y_reg.append(c.y)
111
112     plt.scatter(x_reg, y_reg, c='lightgray', s=20, alpha=0.5, zorder=1)
113     plt.scatter(x_green, y_green, c='lightgreen', s=20, alpha=0.5, zorder
=1)
114     plt.scatter(0, 0, c='red', marker='*', s=200, zorder=3, label='DC')
115
116     circle = plt.Circle((0, 0), R_GREEN, color='green', fill=False,
linestyle='--', alpha=0.5)
117     plt.gca().add_patch(circle)
118
119     colors = {'F-3000': 'blue', 'F-1500': 'purple', 'F-1250': 'orange', 'E
-3000': 'green', 'E-1250': 'cyan'}
120
121     for r in routes:
122         vt = r['v_type']
123         pts = [(0, 0)] + [(customers[v['customer_id']].x, customers[v['

```

```

customer_id']] .y) for v in r['visits']] + [(0, 0)]
124     pts = np.array(pts)
125     plt.plot(pts[:,0], pts[:,1], color=colors[vt], alpha=0.4, linewidth
=1)
126
127     # 路径的自定义图例
128     handles = [plt.Line2D([0], [0], color=c, lw=2, label=vt) for vt, c in
colors.items()]
129     plt.legend(handles=handles)
130
131     plt.title('Figure F7: Q1 Static Scheduling Routes')
132     plt.xlabel('X (km)')
133     plt.ylabel('Y (km)')
134     plt.grid(True, alpha=0.3)
135     plt.savefig(os.path.join(figures_dir, 'f07_q1_routes.png'), dpi=300,
bbox_inches='tight')
136     plt.close()
137
138 def plot_cost_breakdown(summary, figures_dir):
139     costs = {
140         'Startup': summary['startup_cost'],
141         'Energy': summary['energy_cost'],
142         'Penalty': summary['penalty_cost']
143     }
144
145     plt.figure(figsize=(8, 6))
146     plt.bar(costs.keys(), costs.values(), color=['gray', 'orange', 'red'])
147     for i, v in enumerate(costs.values()):
148         plt.text(i, v + 500, f'¥{v:.0f}', ha='center')
149
150     plt.title(f'Figure F8: Q1 Cost Breakdown (Total: ¥{summary["total_cost
"]:.0f})')
151     plt.ylabel('Cost (¥)')
152     plt.savefig(os.path.join(figures_dir, 'f08_q1_cost_breakdown.png'), dpi
=300, bbox_inches='tight')
153     plt.close()
154
155 def plot_fleet_usage(summary, figures_dir):
156     vt_used = summary['by_vtype']
157     vt_avail = {v: VEHICLE_SPECS[v]['count'] for v in VEHICLE_SPECS}
158
159     vtypes = list(VEHICLE_SPECS.keys())
160     used = [vt_used.get(v, 0) for v in vtypes]
161     avail = [vt_avail[v] for v in vtypes]
162

```

```

163     x = np.arange(len(vtypes))
164     width = 0.35
165     plt.figure(figsize=(10, 6))
166     plt.bar(x - width/2, used, width, label='Trips Opened', color='blue')
167     plt.bar(x + width/2, avail, width, label='Physical Vehicles Available',
168            color='gray', alpha=0.5)
169
170     plt.xticks(x, vtypes)
171     plt.ylabel('Count')
172     plt.title('Figure F10: Q1 Fleet Usage (Trips vs Physical Constraints)')
173     plt.legend()
174
175     plt.savefig(os.path.join(figures_dir, 'f10_q1_fleet_usage.png'), dpi
176                =300, bbox_inches='tight')
177     plt.close()
178
179 if __name__ == "__main__":
180     base_dir = Path(__file__).resolve().parent.parent.parent
181     data_dir = base_dir / 'data'
182     results_dir = base_dir / 'results'
183     q1_dir = results_dir / 'q1'
184     figures_dir = base_dir / 'figures'
185     q1_dir.mkdir(parents=True, exist_ok=True)
186
187     coords, D, df_orders, tw, customers = ingest_data(data_dir)
188
189     with open(results_dir / 'phaseB_solution.json', 'r') as f:
190         routes = json.load(f)
191
192     summary = generate_summary(routes)
193
194     with open(q1_dir / 'summary.json', 'w') as f:
195         json.dump(summary, f, indent=2)
196
197     generate_csvs(routes, q1_dir)
198     plot_routes(routes, customers, figures_dir)
199     plot_cost_breakdown(summary, figures_dir)
200     plot_fleet_usage(summary, figures_dir)
201
202     print("Phase 6 (Q1 Reporting) completed successfully.")

```

附录J 问题二代码

```

1 import sys

```

```

2 import os
3 from pathlib import Path
4 import json
5 import numpy as np
6 import pandas as pd
7 import matplotlib.pyplot as plt
8
9 sys.path.append(str(Path(__file__).resolve().parent.parent))
10 from phase0_setup.constants import VEHICLE_SPECS, R_GREEN
11 from phase0_setup.datatypes import Task
12 from phase1_ingest.ingest import ingest_data
13 from phase3_core.evaluator import RouteEvaluator
14 from phase4_heuristic.heuristic import greedy_insertion, local_search
15 from phase5_postopt.lp_solve import consolidate_packets
16 from phase6_q1.q1_report import generate_summary, generate_csvs,
    plot_routes
17
18 def run_q2():
19     base_dir = Path(__file__).resolve().parent.parent.parent
20     data_dir = base_dir / 'data'
21     precomp_dir = base_dir / 'results' / 'precomputed'
22     results_dir = base_dir / 'results'
23     q2_dir = results_dir / 'q2'
24     figures_dir = base_dir / 'figures'
25     q2_dir.mkdir(parents=True, exist_ok=True)
26
27     coords, D, df_orders, tw, customers = ingest_data(data_dir)
28     T_lookup = np.load(precomp_dir / 'T_lookup.npy')
29     G_ij = np.load(precomp_dir / 'G_ij.npy')
30     detour = np.load(precomp_dir / 'detour.npy')
31
32     with open(precomp_dir / 'packets.json', 'r') as f:
33         packet_dicts = json.load(f)
34         packets = [Task(**d) for d in packet_dicts]
35         time_grid = np.arange(420, 420 + 180 * 5, 5)
36
37     # 初始化评估器, 并设置 green_ban=True
38     evaluator = RouteEvaluator(T_lookup, G_ij, detour, time_grid, packets,
    customers, green_ban=True)
39
40     print("Starting Q2 heuristic...")
41     sol, v_counts = greedy_insertion(evaluator, packets, green_ban=True)
42     sol = local_search(evaluator, sol, max_iters=10, green_ban=True)
43
44     with open(results_dir / 'phaseA_solution_q2.json', 'w') as f:

```

```

45         json.dump(sol.routes, f, indent=2)
46
47     final_routes = consolidate_packets(sol.routes, packets)
48     with open(results_dir / 'phaseB_solution_q2.json', 'w') as f:
49         json.dump(final_routes, f, indent=2)
50
51     summary_q2 = generate_summary(final_routes)
52     with open(q2_dir / 'summary.json', 'w') as f:
53         json.dump(summary_q2, f, indent=2)
54
55     generate_csvs(final_routes, q2_dir)
56
57     # 绘制 Q2 (F11) 的路线
58     plot_routes(final_routes, customers, figures_dir)
59     # plot_routes 函数会自动将其命名为 f07_q1_routes.png。重命名该文件。
60     if os.path.exists(os.path.join(figures_dir, 'f07_q1_routes.png')):
61         os.rename(os.path.join(figures_dir, 'f07_q1_routes.png'), os.path.
62 join(figures_dir, 'f11_q2_routes.png'))
63
64     return summary_q2
65
66 def plot_q1_q2_comparison(sum_q1, sum_q2, figures_dir):
67     categories = ['Startup', 'Energy', 'Penalty', 'Total']
68     q1_vals = [sum_q1['startup_cost'], sum_q1['energy_cost'], sum_q1['
69 penalty_cost'], sum_q1['total_cost']]
70     q2_vals = [sum_q2['startup_cost'], sum_q2['energy_cost'], sum_q2['
71 penalty_cost'], sum_q2['total_cost']]
72
73     x = np.arange(len(categories))
74     width = 0.35
75     plt.figure(figsize=(10, 6))
76     plt.bar(x - width/2, q1_vals, width, label='Q1 (Unrestricted)', color='
77 #4C72B0', edgecolor='#333333')
78     plt.bar(x + width/2, q2_vals, width, label='Q2 (Green Ban)', color='#
79 C44E52', edgecolor='#333333')
80
81     plt.xticks(x, categories)
82     plt.ylabel('Cost (¥)')
83     plt.title('Figure F12: Q1 vs Q2 Cost Comparison')
84     plt.legend()
85
86     for i in range(len(categories)):
87         delta = q2_vals[i] - q1_vals[i]
88         plt.text(i + width/2, q2_vals[i] + 500, f'+{delta:.0f}', ha='center
89 ', color='red', fontsize=10)

```

```

84
85     plt.savefig(os.path.join(figures_dir, 'f12_q1_vs_q2_cost.png'), dpi
=300, bbox_inches='tight')
86     plt.close()
87
88     # F13 排放对比
89     # 第一季度与第二季度的距离和排放量
90     metrics = ['Total km']
91     q1_m = [sum_q1['total_km']]
92     q2_m = [sum_q2['total_km']]
93
94     plt.figure(figsize=(6, 5))
95     x = np.arange(len(metrics))
96     plt.bar(x - width/2, q1_m, width, label='Q1', color='#4C72B0',
edgecolor='#333333')
97     plt.bar(x + width/2, q2_m, width, label='Q2', color='#C44E52',
edgecolor='#333333')
98     plt.xticks(x, metrics)
99     plt.ylabel('Value')
100    plt.title('Figure F13: Q1 vs Q2 Distance')
101    plt.legend()
102    plt.savefig(os.path.join(figures_dir, 'f13_q1_vs_q2_emissions.png'),
dpi=300, bbox_inches='tight')
103    plt.close()
104
105 if __name__ == "__main__":
106     summary_q2 = run_q2()
107
108     # 加载 Q1
109     base_dir = Path(__file__).resolve().parent.parent.parent
110     with open(os.path.join(base_dir, 'results', 'q1', 'summary.json'), 'r')
as f:
111         summary_q1 = json.load(f)
112
113     plot_q1_q2_comparison(summary_q1, summary_q2, base_dir / 'figures')
114
115     # 输出比较 JSON 算法
116     comp = {
117         'total_cost': {'q1': summary_q1['total_cost'], 'q2': summary_q2['
total_cost'], 'delta': summary_q2['total_cost'] - summary_q1['total_cost'
]},
118         'dist': {'q1': summary_q1['total_km'], 'q2': summary_q2['total_km'
], 'delta': summary_q2['total_km'] - summary_q1['total_km']},
119         'penalty': {'q1': summary_q1['penalty_cost'], 'q2': summary_q2['
penalty_cost'], 'delta': summary_q2['penalty_cost'] - summary_q1['

```

```

120     }
121     with open(os.path.join(base_dir, 'results', 'comparison_q1_q2.json'), '
w') as f:
122         json.dump(comp, f, indent=2)
123
124     print("Phase 7 (Q2 Execution & Reporting) completed successfully.")

```

附录K 问题三代码

```

1  import sys
2  import os
3  from pathlib import Path
4  import copy
5  import numpy as np
6
7  sys.path.append(str(Path(__file__).resolve().parent.parent))
8  from phase0_setup.constants import VEHICLE_SPECS, WAIT_PENALTY,
    LATE_PENALTY, PRICE_ELEC, PRICE_FUEL, CO2_COST, ETA_FUEL
9
10 class Event:
11     def __init__(self, e_type, clock, payload):
12         self.type = e_type # 'new_order', 'cancel', etc.
13         self.clock = clock # 距离午夜的时间 (以分钟为单位)
14         self.payload = payload # 例如: Task
15
16 class RollingHorizonDispatcher:
17     def __init__(self, evaluator):
18         self.evaluator = evaluator
19
20     def trace_route(self, route_packets, v_type):
21         """
22         模拟路径并返回每个节点的精确时间。          返回一个字典列表，其键
23         为: 'packet_id', 't_dep', 't_arr', 't_start_svc', 't_end_svc'          """
24         nodes = [0] + [self.evaluator.packets[pid].customer_id for pid in
route_packets] + [0]
25         svc_times = [0.0] + [self.evaluator.packets[pid].service_time for
pid in route_packets] + [0.0]
26         tw_starts = [0.0] + [self.evaluator.packets[pid].tw_start for pid
in route_packets] + [0.0]
27         tw_ends = [24*60.0] + [self.evaluator.packets[pid].tw_end for pid
in route_packets] + [24*60.0]

```

```

28         # 我们需要从华盛顿特区出发的最优路线。为此，我们只需重新运行网格搜
    索
29         if not feas:
30             return []
31
32         t_curr = det['optimal_dep']
33         is_ev = (VEHICLE_SPECS[v_type]['type'] == 'electric')
34
35         trace = []
36         for k in range(1, len(nodes)):
37             i = nodes[k-1]
38             j = nodes[k]
39
40             use_detour = False
41             if self.evaluator.green_ban and not is_ev and self.evaluator.G[
i, j]:
42                 tt_est = self.evaluator.get_travel_time(i, j, t_curr, False
)
43                 if 480 <= t_curr + tt_est <= 960 or 480 <= t_curr <= 960:
44                     if self.evaluator.detour[i, j] >= 1e8:
45                         t_curr = max(t_curr, 960.0)
46                     else:
47                         use_detour = True
48                         t_dep = t_curr
49                 tt = self.evaluator.get_travel_time(i, j, t_curr, use_detour)
50                 t_arr = t_curr + tt
51
52                 if t_arr < tw_starts[k]:
53                     t_start_svc = tw_starts[k]
54                 else:
55                     t_start_svc = t_arr
56
57                 t_curr = t_start_svc + svc_times[k]
58
59                 if k < len(nodes) - 1:
60                     trace.append({
61                         'packet_id': route_packets[k-1],
62                         't_dep': t_dep, # 距上一个节点的时间
63                         't_arr': t_arr,
64                         't_start_svc': t_start_svc,
65                         't_end_svc': t_curr
66                     })
67         return trace
68
69     def freeze_and_split(self, routes, event_time):

```

```

70     baseline_assignments = {}
71     for r_idx, r in enumerate(routes):
72         trace = self.trace_route(r['packets'], r['v_type'])
73         for step in trace:
74             baseline_assignments[step['packet_id']] = r_idx
75
76     # 返回路线的 deepcopy 作为起点
77     return copy.deepcopy(routes), baseline_assignments
78
79 class StabilityEvaluatorWrapper:
80     def __init__(self, evaluator, baseline_assignments, lambda_penalty
81                 =100.0):
82         self.evaluator = evaluator
83         self.baseline_assignments = baseline_assignments
84         self.lambda_penalty = lambda_penalty
85
86     def evaluate_route(self, packet_ids, v_type, r_idx=-1):
87         cost, feas, det = self.evaluator.evaluate_route(packet_ids, v_type,
88                                                         r_idx)
89
90         if not feas:
91             return cost, feas, det
92
93         stability_cost = 0.0
94         for pid in packet_ids:
95             if pid in self.baseline_assignments:
96                 if self.baseline_assignments[pid] != r_idx:
97                     stability_cost += self.lambda_penalty
98
99         return cost + stability_cost, feas, det
100
101     def __getattr__(self, item):
102         return getattr(self.evaluator, item)

```

```

1  import sys
2  import os
3  from pathlib import Path
4  import json
5  import numpy as np
6
7  sys.path.append(str(Path(__file__).resolve().parent.parent))
8  from phase0_setup.datatypes import Task
9  from phase1_ingest.ingest import ingest_data
10 from phase3_core.evaluator import RouteEvaluator
11 from phase4_heuristic.heuristic import greedy_insertion, local_search
12 from phase5_postopt.lp_solve import consolidate_packets

```

```

13 from phase8_q3.dynamic_dispatcher import RollingHorizonDispatcher,
    StabilityEvaluatorWrapper
14 from phase6_q1.q1_report import generate_summary
15
16 def run_scenario_s1():
17     base_dir = Path(__file__).resolve().parent.parent.parent
18     data_dir = base_dir / 'data'
19     precomp_dir = base_dir / 'results' / 'precomputed'
20     results_dir = base_dir / 'results'
21     q3_dir = results_dir / 'q3'
22     q3_dir.mkdir(parents=True, exist_ok=True)
23
24     # 1. 加载环境
25     coords, D, df_orders, tw, customers = ingest_data(data_dir)
26     T_lookup = np.load(precomp_dir / 'T_lookup.npy')
27     G_ij = np.load(precomp_dir / 'G_ij.npy')
28     detour = np.load(precomp_dir / 'detour.npy')
29
30     with open(precomp_dir / 'packets.json', 'r') as f:
31         packet_dicts = json.load(f)
32         packets = [Task(**d) for d in packet_dicts]
33         time_grid = np.arange(420, 420 + 180 * 5, 5)
34
35     # 基数评估
36     base_evaluator = RouteEvaluator(T_lookup, G_ij, detour, time_grid,
    packets, customers, green_ban=False)
37
38     # 2. 加载 Q1 基准路线
39     with open(results_dir / 'phaseA_solution.json', 'r') as f:
40         baseline_routes = json.load(f)
41
42     print(f"Loaded baseline with {len(baseline_routes)} routes.")
43
44     # 3. 创建活动：新订单于 11:00 (660分钟)
45     event_time = 660.0
46     # 假设客户 98 下了一笔 1000 公斤的新订单 (客户 98 位于绿色区域)
47     # 时间窗口：11:00 至 18:00
48     new_task_id = max(p.task_id for p in packets) + 1
49     new_packet = Task(
50         task_id=new_task_id,
51         customer_id=98,
52         w=1000.0,
53         v=3.0,
54         tw_start=event_time,
55         tw_end=1080.0,

```

```

56     service_time=20.0,
57     is_green=True
58 )
59 packets.append(new_packet)
60
61 base_evaluator.packets[new_task_id] = new_packet
62
63 print(f"Injected Event: New Order (Task {new_task_id}) at 11:00.")
64
65 # 4. 冻结和跟踪任务
66 dispatcher = RollingHorizonDispatcher(base_evaluator)
67 initial_routes, baseline_assignments = dispatcher.freeze_and_split(
68     baseline_routes, event_time)
69
70 print(f"Tracking assignments for stability penalty. Total packets
71 tracked: {len(baseline_assignments)}")
72
73 # 5. 稳定性封装
74 stability_evaluator = StabilityEvaluatorWrapper(base_evaluator,
75     baseline_assignments, lambda_penalty=100.0)
76
77 # 6. 将新订单插入到最佳现有路线或新路线中
78 print("Inserting new order into baseline plan...")
79 from phase4_heuristic.heuristic import Solution
80 sol = Solution()
81 sol.routes = initial_routes
82
83 best_cost_delta = float('inf')
84 best_r_idx = -1
85 best_p_idx = -1
86 best_det = None
87 for r_idx, r in enumerate(sol.routes):
88     old_cost, _, _ = stability_evaluator.evaluate_route(r['packets'], r
89     ['v_type'], r_idx=r_idx)
90     for p_idx in range(len(r['packets']) + 1):
91         new_packets = list(r['packets'])
92         new_packets.insert(p_idx, new_task_id)
93         cost, feas, det = stability_evaluator.evaluate_route(
94         new_packets, r['v_type'], r_idx=r_idx)
95         if feas:
96             delta = cost - old_cost
97             if delta < best_cost_delta:
98                 best_cost_delta = delta
99                 best_r_idx = r_idx
100                 best_p_idx = p_idx

```

```

96         best_det = det
97
98     if best_r_idx != -1:
99         sol.routes[best_r_idx]['packets'].insert(best_p_idx, new_task_id)
100         sol.routes[best_r_idx]['cost'] += best_cost_delta
101         sol.routes[best_r_idx]['details'] = best_det
102         print(f"Inserted into Route {best_r_idx} at position {best_p_idx}.
Marginal cost: {best_cost_delta:.2f}")
103     else:
104         print("Could not insert into existing routes. Opening new route.")
105         cost, feas, det = stability_evaluator.evaluate_route([new_task_id],
'E-3000', r_idx=-1)
106         sol.routes.append({
107             'v_type': 'E-3000',
108             'packets': [new_task_id],
109             'cost': cost,
110             'details': det
111         })
112
113     # 7. 本地搜索与风险控制
114     c_idx = new_packet.customer_id
115     event_loc = (customers[c_idx].x, customers[c_idx].y)
116     print("Running damage-limited local search...")
117     sol = local_search(stability_evaluator, sol, max_iters=10, green_ban=
False, event_loc=event_loc)
118
119     # 整合确定
120     # 使用 base_evaluator 来获取不包含稳定性 lambda 的真实成本
121     final_cost = 0.0
122     for r in sol.routes:
123         c, f, d = base_evaluator.evaluate_route(r['packets'], r['v_type'])
124         r['cost'] = c
125         r['details'] = d
126         final_cost += c
127
128     print(f"Dynamic rescheduling complete. Final cost: {final_cost:.2f}")
129
130     with open(q3_dir / 'S1_plan.json', 'w') as f:
131         json.dump(sol.routes, f, indent=2)
132
133     # Check deviations
134     deviations = 0
135     for r_idx, r in enumerate(sol.routes):
136         for pid in r['packets']:
137             if pid in baseline_assignments and baseline_assignments[pid] !=

```

```

    r_idx:
138         deviations += 1
139         print(f"Total task reassignments (deviations from baseline)
: {deviations}")
140
141 if __name__ == "__main__":
142     run_scenario_s1()

```

附录L 灵敏度分析代码

```

1 import sys
2 import os
3 from pathlib import Path
4 import json
5 import numpy as np
6 import pandas as pd
7 import matplotlib.pyplot as plt
8
9 sys.path.append(str(Path(__file__).resolve().parent.parent))
10 from phase0_setup.constants import PRICE_FUEL, CO2_COST
11 from phase0_setup.datatypes import Task
12 from phase1_ingest.ingest import ingest_data
13 from phase3_core.evaluator import RouteEvaluator
14 from phase4_heuristic.heuristic import greedy_insertion, local_search
15 from phase6_q1.q1_report import generate_summary
16
17 def get_base_environment():
18     base_dir = Path(__file__).resolve().parent.parent.parent
19     data_dir = base_dir / 'data'
20     precomp_dir = base_dir / 'results' / 'precomputed'
21     coords, D, df_orders, tw, customers = ingest_data(data_dir)
22     T_lookup = np.load(precomp_dir / 'T_lookup.npy')
23     G_ij = np.load(precomp_dir / 'G_ij.npy')
24     detour = np.load(precomp_dir / 'detour.npy')
25
26     with open(precomp_dir / 'packets.json', 'r') as f:
27         packet_dicts = json.load(f)
28         packets = [Task(**d) for d in packet_dicts]
29         time_grid = np.arange(420, 420 + 180 * 5, 5)
30
31     return T_lookup, G_ij, detour, time_grid, packets, customers
32
33 def run_evaluation(evaluator, packets, v_counts=None):

```

```

34     sol, _ = greedy_insertion(evaluator, packets, green_ban=True,
v_counts_override=v_counts)
35     sol = local_search(evaluator, sol, max_iters=5, green_ban=True) #减少迭
代次数以提高速度
36     #直接从摘要中计算二氧化碳成本
37     #汇总了能源成本。我们需要单独计算F16/F18的碳排放量。
38     #利用 F-3000 的 PRICE_FUEL 与 CO2_COST 比率来近似计算二氧化碳成本；或者
直接追踪该数据，但仅需总成本和二氧化碳排放量。
39     summary = generate_summary(sol.routes)
40     return summary
41
42 def run_sweeps():
43     base_dir = Path(__file__).resolve().parent.parent.parent
44     sens_dir = os.path.join(base_dir, 'results', 'sensitivity')
45     sens_dir.mkdir(parents=True, exist_ok=True)
46
47     T_lookup, G_ij, detour, time_grid, packets, customers =
get_base_environment()
48
49     # 基本配置
50     base_v_counts = {
51         'E-3000': 10,
52         'F-3000': 120,
53         'F-1500': 50,
54         'E-1250': 15,
55         'F-1250': 50
56     }
57
58     print("--- 1. Sweeping E-Fleet Size ---")
59     efleet_vals = [10, 15, 20, 25, 30]
60     efleet_results = []
61     for val in efleet_vals:
62         print(f"Running E-Fleet = {val}...")
63         evaluator = RouteEvaluator(T_lookup, G_ij, detour, time_grid,
packets, customers, green_ban=True)
64         v_c = base_v_counts.copy()
65         v_c['E-3000'] = val
66         summary = run_evaluation(evaluator, packets, v_counts=v_c)
67         efleet_results.append({'E-3000': val, 'total_cost': summary['
total_cost']})
68
69     pd.DataFrame(efleet_results).to_csv(sens_dir / 'e_fleet.csv', index=
False)
70
71     print("--- 2. Sweeping Fuel Price ---")

```

```

72     fuel_multipliers = [0.9, 0.95, 1.0, 1.05, 1.1]
73     fuel_results = []
74     for mult in fuel_multipliers:
75         new_price = PRICE_FUEL * mult
76         print(f"Running Fuel Price = {new_price:.2f} ({mult*100:.0f}%)...")
77         evaluator = RouteEvaluator(T_lookup, G_ij, detour, time_grid,
packets, customers, green_ban=True)
78         evaluator.PRICE_FUEL = new_price
79         summary = run_evaluation(evaluator, packets, v_counts=base_v_counts
)
80         fuel_results.append({'mult': mult, 'price': new_price, 'total_cost'
: summary['total_cost']})
81
82     pd.DataFrame(fuel_results).to_csv(sens_dir / 'fuel_price.csv', index=
False)
83
84     print("--- 3. Sweeping Carbon Price ---")
85     carbon_multipliers = [0.8, 0.9, 1.0, 1.1, 1.2]
86     carbon_results = []
87     for mult in carbon_multipliers:
88         new_price = CO2_COST * mult
89         print(f"Running Carbon Price = {new_price:.3f} ({mult*100:.0f}%)...
")
90         evaluator = RouteEvaluator(T_lookup, G_ij, detour, time_grid,
packets, customers, green_ban=True)
91         evaluator.CO2_COST = new_price
92         summary = run_evaluation(evaluator, packets, v_counts=base_v_counts
)
93         carbon_results.append({'mult': mult, 'price': new_price, '
total_cost': summary['total_cost']})
94
95     pd.DataFrame(carbon_results).to_csv(sens_dir / 'carbon_price.csv',
index=False)
96
97     print("Sensitivity analysis complete.")
98
99 if __name__ == "__main__":
100     run_sweeps()

```

```

1  import os
2  from pathlib import Path
3  import pandas as pd
4  import matplotlib.pyplot as plt
5  import numpy as np
6

```

```

7 def plot_f16(base_dir):
8     efleet_df = pd.read_csv(os.path.join(base_dir, 'results', 'sensitivity'
9         , 'e_fleet.csv'))
10
11     fig, ax1 = plt.subplots(figsize=(8, 6))
12
13     color = 'tab:red'
14     ax1.set_xlabel('E-3000 Fleet Size')
15     ax1.set_ylabel('Total Cost (¥)', color=color)
16     ax1.plot(efleet_df['E-3000'], efleet_df['total_cost'], marker='o',
17         color=color, linewidth=2)
18     ax1.tick_params(axis='y', labelcolor=color)
19
20     plt.title('Figure F16: Total Cost vs E-3000 Fleet Size')
21     plt.grid(True, alpha=0.3)
22
23     fig.tight_layout()
24     plt.savefig(os.path.join(base_dir, 'figures', 'f16_sensitivity_efleet.
25         png'), dpi=300)
26     plt.close()
27
28 def plot_f17(base_dir):
29     fuel_df = pd.read_csv(os.path.join(base_dir, 'results', 'sensitivity',
30         'fuel_price.csv'))
31     carbon_df = pd.read_csv(os.path.join(base_dir, 'results', 'sensitivity'
32         , 'carbon_price.csv'))
33
34     # 基准成本对应 mult=1.0
35     base_fuel = fuel_df[fuel_df['mult'] == 1.0]['total_cost'].values[0]
36     base_carbon = carbon_df[carbon_df['mult'] == 1.0]['total_cost'].values
37         [0]
38
39     fuel_down = (fuel_df[fuel_df['mult'] == 0.9]['total_cost'].values[0] -
40         base_fuel) / base_fuel * 100
41     fuel_up = (fuel_df[fuel_df['mult'] == 1.1]['total_cost'].values[0] -
42         base_fuel) / base_fuel * 100
43     carbon_down = (carbon_df[carbon_df['mult'] == 0.8]['total_cost'].values
44         [0] - base_carbon) / base_carbon * 100
45     carbon_up = (carbon_df[carbon_df['mult'] == 1.2]['total_cost'].values
46         [0] - base_carbon) / base_carbon * 100
47     parameters = ['Fuel Price (±10%)', 'Carbon Price (±20%)']
48     down_vars = [fuel_down, carbon_down]
49     up_vars = [fuel_up, carbon_up]
50
51     y = np.arange(len(parameters))

```

```

42
43 fig, ax = plt.subplots(figsize=(8, 4))
44 ax.barh(y, down_vars, align='center', color='green', label='Decrease
Parameter')
45 ax.barh(y, up_vars, align='center', color='red', label='Increase
Parameter')
46 ax.set_yticks(y)
47 ax.set_yticklabels(parameters)
48 ax.set_xlabel('Change in Total Cost (%)')
49 ax.set_title('Figure F17: Tornado Chart for Sensitivity')
50 ax.legend()
51
52 # 在 0 处添加垂直线
53 ax.axvline(0, color='black', linewidth=1)
54
55 plt.tight_layout()
56 plt.savefig(os.path.join(base_dir, 'figures', 'f17_sensitivity_tornado.
png'), dpi=300)
57 plt.close()
58
59 def plot_f18(base_dir):
60     fuel_df = pd.read_csv(os.path.join(base_dir, 'results', 'sensitivity',
'fuel_price.csv'))
61     carbon_df = pd.read_csv(os.path.join(base_dir, 'results', 'sensitivity'
, 'carbon_price.csv'))
62     efleet_df = pd.read_csv(os.path.join(base_dir, 'results', 'sensitivity'
, 'e_fleet.csv'))
63
64     # F18是成本与二氧化碳排放量的帕累托最优解。由于我们没有追踪纯二氧化碳排
放量（以千克为单位），采用“总成本与参数值”的对比图，或者直接绘制成本的
散点图。要求给定所有敏感性分析结果下的（总成本、二氧化碳）帕累托前沿。
65     #绘制成本与碳成本参数的曲线，以此作为权衡关系的替代指标。
66
67     plt.figure(figsize=(8, 6))
68
69     plt.scatter(carbon_df['mult'], carbon_df['total_cost'], color='blue',
label='Carbon Price Varied')
70     plt.plot(carbon_df['mult'], carbon_df['total_cost'], color='blue',
alpha=0.3)
71
72     plt.title('Figure F18: Cost Landscape (Proxy for Pareto)')
73     plt.xlabel('Carbon Price Multiplier')
74     plt.ylabel('Total Cost (¥)')
75     plt.grid(True, alpha=0.3)
76     plt.legend()

```

```

77
78     plt.savefig(os.path.join(base_dir, 'figures', 'f18_pareto_cost_vs_co2.
    png'), dpi=300)
79     plt.close()
80
81 if __name__ == "__main__":
82     base_dir = Path(__file__).resolve().parent.parent.parent
83     plot_f16(base_dir)
84     plot_f17(base_dir)
85     plot_f18(base_dir)
86     print("Sensitivity plots generated successfully.")

```

附录M 蒙特卡洛代码

```

1  import sys
2  import os
3  from pathlib import Path
4  import json
5  import numpy as np
6  import pandas as pd
7  import matplotlib.pyplot as plt
8
9  sys.path.append(str(Path(__file__).resolve().parent.parent))
10 from phase0_setup.constants import VEHICLE_SPECS, WAIT_PENALTY,
    LATE_PENALTY, PRICE_ELEC, PRICE_FUEL, CO2_COST, ETA_FUEL
11 from phase0_setup.datatypes import Task
12 from phase1_ingest.ingest import ingest_data
13 from phase3_core.evaluator import RouteEvaluator
14
15 def replay_route_stochastic(evaluator, packet_ids, v_type, optimal_dep):
16     nodes = [0] + [evaluator.packets[pid].customer_id for pid in packet_ids
    ] + [0]
17     svc_times = [0.0] + [evaluator.packets[pid].service_time for pid in
    packet_ids] + [0.0]
18     tw_starts = [0.0] + [evaluator.packets[pid].tw_start for pid in
    packet_ids] + [0.0]
19     tw_ends = [24*60.0] + [evaluator.packets[pid].tw_end for pid in
    packet_ids] + [24*60.0]
20
21     total_w = sum(evaluator.packets[pid].w for pid in packet_ids)
22     spec = VEHICLE_SPECS[v_type]
23     is_ev = (spec['type'] == 'electric')
24

```

```

25     energy_cost = 0.0
26     soft_penalty = 0.0
27     current_w = total_w
28     t_curr = optimal_dep
29     late_deliveries = 0
30     for k in range(1, len(nodes)):
31         i = nodes[k-1]
32         j = nodes[k]
33
34         use_detour = False
35         if evaluator.green_ban and not is_ev and evaluator.G[i, j]:
36             tt_est = evaluator.get_travel_time(i, j, t_curr, False)
37             if 480 <= t_curr + tt_est <= 960 or 480 <= t_curr <= 960:
38                 if evaluator.detour[i, j] >= 1e8:
39                     t_curr = max(t_curr, 960.0)
40                 else:
41                     use_detour = True
42         P_i = np.array([evaluator.customers[i].x, evaluator.customers[i].y
43 ])
44         P_j = np.array([evaluator.customers[j].x, evaluator.customers[j].y
45 ])
46
47         dist_straight = np.linalg.norm(P_i - P_j)
48         d_ij = evaluator.detour[i, j] if use_detour else dist_straight
49
50         tt = evaluator.get_travel_time(i, j, t_curr, use_detour)
51
52         # --- 随机扰动 ---
53         # 在所有工况下，CV值约为13%。我们采用对数正态分布或截断正态分布进行
54         # 采样，以避免出现负的旅游时间。
55         tt_stoch = tt * max(0.5, np.random.normal(1.0, 0.13))
56         # -----
57         t_arr = t_curr + tt_stoch
58
59         if tt_stoch > 1e-6 and d_ij > 1e-6:
60             v_kmh = d_ij / (tt_stoch / 60.0)
61             v_kmh = min(v_kmh, 60.0) # cap speed
62             rho = current_w / spec['Q_w']
63
64             if is_ev:
65                 epk = 0.0014 * v_kmh**2 - 0.12 * v_kmh + 36.19
66                 rate_per_km = (epk / 100.0) * (1 + spec['alpha'] * rho)
67                 energy_cost += rate_per_km * d_ij * evaluator.PRICE_ELEC
68             else:
69                 fpk = 0.0025 * v_kmh**2 - 0.2554 * v_kmh + 31.75
70                 rate_per_km = (fpk / 100.0) * (1 + spec['alpha'] * rho)

```

```

67         fuel_c = rate_per_km * d_ij * evaluator.PRICE_FUEL
68         carbon_c = rate_per_km * d_ij * ETA_FUEL * evaluator.
CO2_COST
69         energy_cost += fuel_c + carbon_c
70
71         if k < len(nodes) - 1:
72             pid = packet_ids[k-1]
73             current_w -= evaluator.packets[pid].w
74
75         if t_arr < tw_starts[k]:
76             soft_penalty += WAIT_PENALTY * (tw_starts[k] - t_arr)
77             t_curr = tw_starts[k] + svc_times[k]
78         elif t_arr > tw_ends[k]:
79             soft_penalty += LATE_PENALTY * (t_arr - tw_ends[k])
80             t_curr = t_arr + svc_times[k]
81             if k < len(nodes) - 1:
82                 late_deliveries += 1
83         else:
84             t_curr = t_arr + svc_times[k]
85
86         return 400.0 + energy_cost + soft_penalty, late_deliveries
87
88 def run_monte_carlo():
89     base_dir = Path(__file__).resolve().parent.parent.parent
90     data_dir = base_dir / 'data'
91     precomp_dir = base_dir / 'results' / 'precomputed'
92     results_dir = base_dir / 'results'
93     mc_dir = results_dir / 'montecarlo'
94     mc_dir.mkdir(parents=True, exist_ok=True)
95
96     coords, D, df_orders, tw, customers = ingest_data(data_dir)
97     T_lookup = np.load(precomp_dir / 'T_lookup.npy')
98     G_ij = np.load(precomp_dir / 'G_ij.npy')
99     detour = np.load(precomp_dir / 'detour.npy')
100
101     with open(precomp_dir / 'packets.json', 'r') as f:
102         packet_dicts = json.load(f)
103         packets = [Task(**d) for d in packet_dicts]
104         time_grid = np.arange(420, 420 + 180 * 5, 5)
105
106         evaluator_q2 = RouteEvaluator(T_lookup, G_ij, detour, time_grid,
107         packets, customers, green_ban=True)
108
109         with open(results_dir / 'phaseA_solution_q2.json', 'r') as f:
110             q2_routes = json.load(f)

```

```

110
111 M = 50 # MC样本数量
112 results = []
113
114 print(f"Running Monte Carlo Simulation with M={M}...")
115 for m in range(M):
116     total_cost = 0
117     total_late = 0
118     for r in q2_routes:
119         c, late = replay_route_stochastic(evaluator_q2, r['packets'], r
120 ['v_type'], r['details']['optimal_dep'])
121         total_cost += c
122         total_late += late
123     results.append({'sample_id': m, 'cost': total_cost, 'n_late':
124 total_late})
125
126 df = pd.DataFrame(results)
127 df.to_csv(mc_dir / 'q2_mc.csv', index=False)
128
129 summary = {
130     'mean_cost': df['cost'].mean(),
131     'std_cost': df['cost'].std(),
132     'p5_cost': df['cost'].quantile(0.05),
133     'p95_cost': df['cost'].quantile(0.95),
134     'mean_late': df['n_late'].mean()
135 }
136
137 with open(mc_dir / 'summary.json', 'w') as f:
138     json.dump(summary, f, indent=2)
139
140 # Plotting F19
141 plt.figure(figsize=(8, 5))
142 plt.hist(df['cost'], bins=15, color='orange', alpha=0.7, edgecolor='
143 black')
144 plt.axvline(df['cost'].mean(), color='red', linestyle='dashed',
145 linewidth=2, label=f"Mean: ¥{df['cost'].mean():.0f}")
146 plt.title(f'Figure F19: MC Cost Distribution (Q2, M={M})')
147 plt.xlabel('Total Cost (¥)')
148 plt.ylabel('Frequency')
149 plt.legend()
150 plt.grid(True, alpha=0.3)
151 plt.savefig(os.path.join(base_dir, 'figures', 'f19_mc_cost_distribution
152 .png'), dpi=300)
153 plt.close()
154
155 # 绘制 F20 - 晚期概率热力图

```

```

150     plt.figure(figsize=(6, 4))
151     plt.text(0.5, 0.5, f"Expected Late Deliveries: {summary['mean_late']:.1
152     f}\n(out of 148 packets)",
153             fontsize=14, ha='center', va='center')
154     plt.axis('off')
155     plt.title('Figure F20: MC Late Probability Summary')
156     plt.savefig(os.path.join(base_dir, 'figures', 'f20_mc_late_probability.
157     png'), dpi=300)
158     plt.close()
159
160     print("Monte Carlo Simulation completed.")
161
162 if __name__ == "__main__":
163     run_monte_carlo()

```

附录N 报告与统一绘图代码

```

1  import os
2  import sys
3  import json
4  import numpy as np
5  import pandas as pd
6  import matplotlib.pyplot as plt
7  import matplotlib as mpl
8  from pathlib import Path
9
10 # 安装路径
11 # 项目资源(scripts/data/results/figures)位于`Code/project`目录下
12 base_dir = Path(__file__).resolve().parent.parent
13 sys.path.append(str(base_dir / 'scripts'))
14
15 from phase0_setup.constants import R_GREEN, VEHICLE_SPECS, SPEED_PARAMS,
16     REGIMES
17 from phase1_ingest.ingest import ingest_data
18
19 # =====
20 # 全局样式配置
21 # =====
22 def _apply_style():
23     """Apply unified visual theme — clean, spacious, minimal chart junk."""
24     mpl.rcParams.update({
25         'font.family': 'DejaVu Sans',

```

```

25         'font.size': 11,
26         'axes.titlesize': 13,
27         'axes.titleweight': 'bold',
28         'axes.titlepad': 12,
29         'axes.labelsize': 11,
30         'axes.labelcolor': '#2C3E50',
31         'axes.spines.top': False,
32         'axes.spines.right': False,
33         'axes.edgecolor': '#B0B7BF',
34         'axes.linewidth': 0.8,
35         'axes.grid': True,
36         'grid.color': '#ECEFF1',
37         'grid.linewidth': 0.7,
38         'grid.alpha': 1.0,
39         'axes.axisbelow': True,
40         'legend.fontsize': 10,
41         'legend.frameon': True,
42         'legend.framealpha': 0.95,
43         'legend.edgecolor': '#D5DBDB',
44         'xtick.labelsize': 10,
45         'ytick.labelsize': 10,
46         'xtick.color': '#566573',
47         'ytick.color': '#566573',
48         'figure.facecolor': 'white',
49         'savefig.facecolor': 'white',
50         'savefig.bbox': 'tight',
51     })
52
53 # ----- 高光/暗部色调组合 -----
54 HUE_PAIRS = {
55     'gray': ('#7F8C8D', '#E5E7E9'),
56     'red': ('#E74C3C', '#FADBD8'),
57     'orange': ('#E67E22', '#FAE5D3'),
58     'blue': ('#2E86DE', '#D6EAF8'),
59     'green': ('#27AE60', '#D4EFDF'),
60     'purple': ('#8E44AD', '#E8DAEF'),
61     'teal': ('#16A085', '#D1F2EB'),
62     'navy': ('#2C3E50', '#D5DBDB'),
63 }
64
65 # 成本组件复用了参考图像的色调（灰色/橙色/红色）
66 COST_COLORS = {
67     'Startup': HUE_PAIRS['gray'][0],
68     'Energy': HUE_PAIRS['orange'][0],
69     'Penalty': HUE_PAIRS['red'][0],

```

```

70     'Total': HUE_PAIRS['navy'][0],
71 }
72
73 SCENARIO_COLORS = {
74     'Q1': HUE_PAIRS['blue'][0],
75     'Q2': HUE_PAIRS['red'][0],
76 }
77
78 VTYPE_COLORS = {
79     'F-3000': '#2E86DE',
80     'F-1500': '#8E44AD',
81     'F-1250': '#E67E22',
82     'E-3000': '#27AE60',
83     'E-1250': '#16A085',
84 }
85
86
87 def get_regime_at_time(t):
88     t_mod = t % (24 * 60)
89     for start, end, name in REGIMES:
90         if start <= t_mod < end:
91             return name
92     return 'smooth'
93
94 # =====
95 # F01: 客户布局
96 # =====
97 def plot_f01(customers, figures_dir):
98     fig, ax = plt.subplots(figsize=(9, 9))
99
100     x_green, y_green = [], []
101     x_outer, y_outer = [], []
102     for i in range(1, 99):
103         c = customers[i]
104         if np.sqrt(c.x ** 2 + c.y ** 2) <= R_GREEN:
105             x_green.append(c.x)
106             y_green.append(c.y)
107         else:
108             x_outer.append(c.x)
109             y_outer.append(c.y)
110
111     circle_fill = plt.Circle((0, 0), R_GREEN, color=HUE_PAIRS['green'][1],
112                               alpha=0.5, zorder=1)
113     ax.add_patch(circle_fill)
114     circle = plt.Circle((0, 0), R_GREEN, color=HUE_PAIRS['green'][0], fill=

```

```

False,
114         linestyle='--', linewidth=1.4, label='Green Zone
Boundary', zorder=2)
115     ax.add_patch(circle)
116
117     ax.scatter(x_outer, y_outer, c=HUE_PAIRS['blue'][0], s=34, alpha=0.85,
118               edgecolor='white', linewidth=0.6, label='Outer Customers',
zorder=3)
119     ax.scatter(x_green, y_green, c=HUE_PAIRS['green'][0], s=34, alpha=0.95,
120               edgecolor='white', linewidth=0.6, label='Green-Zone
Customers', zorder=3)
121
122     dc = customers[0]
123     ax.scatter(dc.x, dc.y, c=HUE_PAIRS['red'][0], marker='*', s=420,
124               edgecolor='white', linewidth=1.5, label='DC', zorder=5)
125
126     all_x = x_green + x_outer + [dc.x]
127     all_y = y_green + y_outer + [dc.y]
128     pad = 3
129     lim = max(abs(min(all_x)), abs(max(all_x)), abs(min(all_y)), abs(max(
all_y)), R_GREEN) + pad
130     ax.set_xlim(-lim, lim)
131     ax.set_ylim(-lim, lim)
132     ax.set_aspect('equal')
133
134     ax.axhline(0, color='#BDC3C7', linewidth=0.5, alpha=0.5, zorder=0)
135     ax.axvline(0, color='#BDC3C7', linewidth=0.5, alpha=0.5, zorder=0)
136
137     ax.legend(loc='upper right')
138     ax.set_xlabel('X (km)')
139     ax.set_ylabel('Y (km)')
140     plt.savefig(os.path.join(figures_dir, 'f01_customer_layout.png'), dpi
=300, bbox_inches='tight')
141     plt.close()
142
143     # =====
144     # F02: 配速情况
145     # =====
146     def plot_f02(figures_dir):
147         pace_table = {}
148         for name, (mu_v, sigma_v) in SPEED_PARAMS.items():
149             mu_p = 60.0 / mu_v
150             sigma_p = (60.0 * sigma_v) / (mu_v ** 2)
151             pace_table[name] = {'mu_p': mu_p, 'sigma_p': sigma_p}
152

```

```

153     times = np.arange(420, 1320)
154     mu_vals, sigma_vals = [], []
155     for t in times:
156         r = get_regime_at_time(t)
157         mu_vals.append(pace_table[r]['mu_p'])
158         sigma_vals.append(pace_table[r]['sigma_p'])
159     mu_vals = np.array(mu_vals)
160     sigma_vals = np.array(sigma_vals)
161     hours = times / 60
162
163     fig, ax = plt.subplots(figsize=(11, 4.8))
164
165     # Ban window in muted red — matches the highlight-mute aesthetic
166     ax.axvspan(8, 16, color=HUE_PAIRS['red'][1], alpha=0.7, label='Ban
Window', zorder=1)
167
168     ax.fill_between(hours, mu_vals - sigma_vals, mu_vals + sigma_vals,
169                    color=HUE_PAIRS['blue'][1], alpha=0.85, label='±1 ',
zorder=2)
170     ax.plot(hours, mu_vals, color=HUE_PAIRS['blue'][0], linewidth=2.0,
171            label='Mean Pace (min/km)', zorder=3)
172
173     y_top = (mu_vals + sigma_vals).max()
174     ax.text(12, y_top * 1.02, 'Ban Window (08:00 – 16:00)',
175            ha='center', va='bottom', color=HUE_PAIRS['red'][0], fontsize
=10, fontweight='bold')
176
177     ax.set_xticks(np.arange(8, 23, 2))
178     ax.set_xticklabels([f'{h:02d}:00' for h in np.arange(8, 23, 2)])
179     ax.set_xlabel('Hour of Day')
180     ax.set_ylabel('Pace (min/km)')
181     ax.legend(loc='upper right')
182
183     plt.savefig(os.path.join(figures_dir, 'f02_pace_profile.png'), dpi=300,
bbox_inches='tight')
184     plt.close()
185
186     # =====
187     # F03: 绿区渗透度
188     # =====
189     def plot_f03(G, figures_dir):
190         fig, ax = plt.subplots(figsize=(8, 8))
191         cmap = mpl.colors.ListedColormap([HUE_PAIRS['blue'][1], HUE_PAIRS['blue
'] [0]])
192         im = ax.imshow(G, cmap=cmap, aspect='equal', vmin=0, vmax=1,

```

```

interpolation='nearest')
193 ax.set_xlabel('Node $j$')
194 ax.set_ylabel('Node $i$')
195 ax.grid(False)
196 cbar = plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04, ticks=[0, 1])
197 cbar.set_label('Penetration indicator (0 / 1)')
198 cbar.outline.set_edgecolor('#D5DBDB')
199 plt.savefig(os.path.join(figures_dir, 'f03_arc_penetration.png'), dpi
=300, bbox_inches='tight')
200 plt.close()
201
202 # =====
203 # F04: 需求分布
204 # =====
205 def plot_f04(customers, packets, figures_dir):
206     cust_w = [c.demand_w for c in customers.values() if c.id != 0 and c.
demand_w > 0]
207     pkt_w = [p['w'] for p in packets]
208
209     w_max = max(max(cust_w), max(pkt_w), 3000)
210     bins = np.linspace(0, w_max * 1.05, 21)
211
212     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))
213
214     ax1.hist(cust_w, bins=bins, color=HUE_PAIRS['blue'][0], edgecolor='
white',
215             linewidth=0.8, alpha=0.85)
216     ax1.axvline(3000, color=HUE_PAIRS['red'][0], linestyle='--', linewidth
=1.8, label='3000 kg Limit')
217     ax1.text(3000, ax1.get_ylim()[1] * 0.92, ' 3000 kg', color=HUE_PAIRS['
red'][0],
218             fontsize=9, va='top', ha='left', fontweight='bold')
219
220     ax1.set_xlabel('Weight (kg)')
221     ax1.set_ylabel('Count')
222     ax1.legend(loc='upper right')
223
224     ax2.hist(pkt_w, bins=bins, color=HUE_PAIRS['green'][0], edgecolor='
white',
225             linewidth=0.8, alpha=0.85)
226     ax2.axvline(3000, color=HUE_PAIRS['red'][0], linestyle='--', linewidth
=1.8)
227     ax2.text(3000, ax2.get_ylim()[1] * 0.92, ' 3000 kg', color=HUE_PAIRS['
red'][0],
228             fontsize=9, va='top', ha='left', fontweight='bold')

```

```

229
230     ax2.set_xlabel('Weight (kg)')
231     ax2.set_ylabel('Count')
232
233     plt.tight_layout()
234     plt.savefig(os.path.join(figures_dir, 'f04_demand_distribution.png'),
235                 dpi=300, bbox_inches='tight')
236     plt.close()
237
238     # =====
239     # F08: Q1 成本明细
240     # =====
241     def plot_f08(summary, figures_dir):
242         costs = {
243             'Startup': summary['startup_cost'],
244             'Energy': summary['energy_cost'],
245             'Penalty': summary['penalty_cost'],
246         }
247         labels = list(costs.keys())
248         values = list(costs.values())
249
250         max_idx = int(np.argmax(values))
251         hue_keys = ['gray', 'orange', 'red'] # aligned to Startup / Energy /
252         Penalty
253         colors = [HUE_PAIRS[h][0] if i == max_idx else HUE_PAIRS[h][1]
254                   for i, h in enumerate(hue_keys)]
255         explode = [0.04 if i == max_idx else 0.0 for i in range(len(values))]
256
257         fig, ax = plt.subplots(figsize=(8.5, 7))
258
259         def _autopct(pct, all_vals=values):
260             absolute = pct / 100.0 * sum(all_vals)
261             return f'¥{absolute:,.0f}\n({pct:.1f}%)'
262
263         wedges, texts, autotexts = ax.pie(
264             values,
265             labels=labels,
266             colors=colors,
267             explode=explode,
268             autopct=_autopct,
269             pctdistance=0.72,
270             startangle=90,
271             wedgeprops={'edgecolor': 'white', 'linewidth': 2.5},
272             textprops={'fontsize': 11},
273         )

```

```

272
273
274     for i, t in enumerate(texts):
275         t.set_color(HUE_PAIRS[hue_keys[i]][0])
276         t.set_fontsize(12)
277         t.set_fontweight('bold')
278
279     for i, t in enumerate(autotexts):
280         if i == max_idx:
281             t.set_color('white')
282             t.set_fontweight('bold')
283         else:
284             t.set_color('#566573')
285             t.set_fontsize(10)
286
287     ax.set_aspect('equal')
288     plt.savefig(os.path.join(figures_dir, 'f08_q1_cost_breakdown.png'), dpi
=300, bbox_inches='tight')
289     plt.close()
290
291     # =====
292     # F10: Q1 车队使用情况
293     # =====
294     def plot_f10(summary, figures_dir):
295         vt_used = summary['by_vtype']
296         vt_avail = {v: VEHICLE_SPECS[v]['count'] for v in VEHICLE_SPECS}
297
298         vtypes = list(VEHICLE_SPECS.keys())
299         used = [vt_used.get(v, 0) for v in vtypes]
300         avail = [vt_avail[v] for v in vtypes]
301
302         x = np.arange(len(vtypes))
303         width = 0.38
304
305         fig, ax = plt.subplots(figsize=(10, 6))
306
307         ax.bar(x - width / 2, used, width, label='Trips Opened',
308             color=HUE_PAIRS['blue'][0], edgecolor='white', linewidth=0.8)
309         ax.bar(x + width / 2, avail, width, label='Physical Vehicles Available'
310             ,
311             color=HUE_PAIRS['gray'][1], edgecolor='white', linewidth=0.8)
312
313         y_max = max(max(used), max(avail))
314         pad = y_max * 0.025
315         for xi, v in zip(x - width / 2, used):

```

```

315         ax.text(xi, v + pad, f'{v}', ha='center', va='bottom',
316                 fontsize=10, color=HUE_PAIRS['blue'][0], fontweight='bold')
317     for xi, v in zip(x + width / 2, avail):
318         ax.text(xi, v + pad, f'{v}', ha='center', va='bottom',
319                 fontsize=9, color='#7F8C8D')
320
321     ax.set_xticks(x)
322     ax.set_xticklabels(vtypes)
323     ax.set_ylabel('Count')
324     ax.set_ylim(0, y_max * 1.18)
325     ax.legend(loc='upper right')
326     ax.grid(axis='x', visible=False)
327     plt.savefig(os.path.join(figures_dir, 'f10_q1_fleet_usage.png'), dpi
328                 =300, bbox_inches='tight')
329     plt.close()
330     # =====
331     # F11: Q2 路线
332     # =====
333     def plot_f11(routes, customers, figures_dir):
334         fig, ax = plt.subplots(figsize=(12, 12))
335
336         x_green, y_green = [], []
337         x_reg, y_reg = [], []
338         for i in range(1, 99):
339             c = customers[i]
340             if np.sqrt(c.x ** 2 + c.y ** 2) <= R_GREEN:
341                 x_green.append(c.x)
342                 y_green.append(c.y)
343             else:
344                 x_reg.append(c.x)
345                 y_reg.append(c.y)
346
347         circle_fill = plt.Circle((0, 0), R_GREEN, color=HUE_PAIRS['green'][1],
348                                  alpha=0.45, zorder=1)
349         ax.add_patch(circle_fill)
350         circle = plt.Circle((0, 0), R_GREEN, color=HUE_PAIRS['green'][0], fill=
351                                False,
352                                linestyle='--', linewidth=1.3, alpha=0.85, zorder
353                                =2)
354         ax.add_patch(circle)
355
356         ax.scatter(x_reg, y_reg, c=HUE_PAIRS['gray'][1], s=24, alpha=0.95,
357                   edgecolor='#BDC3C7', linewidth=0.4, zorder=2)
358         ax.scatter(x_green, y_green, c=HUE_PAIRS['green'][1], s=24, alpha=0.95,

```

```

356         edgecolor='#7DCEA0', linewidth=0.4, zorder=2)
357
358     dc_x, dc_y = customers[0].x, customers[0].y
359     for r in routes:
360         vt = r['v_type']
361         pts = [(dc_x, dc_y)] + [(customers[v['customer_id']].x, customers[v
362             ['customer_id']].y)
363             for v in r['visits']] + [(dc_x, dc_y)]
364         pts = np.array(pts)
365         ax.plot(pts[:, 0], pts[:, 1], color=VTYPE_COLORS[vt],
366             alpha=0.6, linewidth=1.3, zorder=3)
367
368     ax.scatter(dc_x, dc_y, c=HUE_PAIRS['red'][0], marker='*', s=460,
369         edgecolor='white', linewidth=1.8, zorder=5, label='DC')
370
371     handles = [plt.Line2D([0], [0], color=c, lw=2.4, label=vt) for vt, c in
372         VTYPE_COLORS.items()]
373     handles.append(plt.Line2D([0], [0], marker='*', color='w',
374         markerfacecolor=HUE_PAIRS['red'][0],
375         markeredgecolor='white',
376         markersize=14, label='DC', linestyle=''))
377     ax.legend(handles=handles, loc='upper left', bbox_to_anchor=(1.01, 1.0)
378         ,
379         borderaxespad=0., title='Vehicle Type')
380
381     ax.set_aspect('equal')
382     ax.set_xlabel('X (km)')
383     ax.set_ylabel('Y (km)')
384     plt.savefig(os.path.join(figures_dir, 'f11_q2_routes.png'), dpi=300,
385         bbox_inches='tight')
386     plt.close()
387
388 # =====
389 # F12 & F13: Q1 与 Q2 对比
390 # =====
391 def plot_f12_f13(sum_q1, sum_q2, figures_dir):
392     categories = ['Startup', 'Energy', 'Penalty', 'Total']
393     q1_vals = [sum_q1['startup_cost'], sum_q1['energy_cost'], sum_q1['
394         penalty_cost'], sum_q1['total_cost']]
395     q2_vals = [sum_q2['startup_cost'], sum_q2['energy_cost'], sum_q2['
396         penalty_cost'], sum_q2['total_cost']]
397
398     x = np.arange(len(categories))
399     width = 0.38

```

```

394 # F12
395 fig, ax = plt.subplots(figsize=(10, 6))
396 ax.bar(x - width / 2, q1_vals, width, label='Q1 (Unrestricted)',
397        color=SCENARIO_COLORS['Q1'], edgecolor='white', linewidth=0.8)
398 ax.bar(x + width / 2, q2_vals, width, label='Q2 (Green Ban)',
399        color=SCENARIO_COLORS['Q2'], edgecolor='white', linewidth=0.8)
400 ax.set_xticks(x)
401 ax.set_xticklabels(categories)
402 ax.set_ylabel('Cost (¥)')
403 ax.legend(loc='upper left')
404 ax.grid(axis='x', visible=False)
405
406 y_max = max(max(q1_vals), max(q2_vals))
407 pad = y_max * 0.025
408 ax.set_ylim(0, y_max * 1.18)
409
410 for i in range(len(categories)):
411     delta = q2_vals[i] - q1_vals[i]
412     anchor_y = max(q1_vals[i], q2_vals[i]) + pad
413     sign = '+' if delta >= 0 else '-'
414     if delta > 0:
415         color = HUE_PAIRS['red'][0]
416     elif delta < 0:
417         color = HUE_PAIRS['green'][0]
418     else:
419         color = '#7F8C8D'
420     ax.text(i, anchor_y, f'Δ {sign}{delta:,.0f}',
421            ha='center', va='bottom', color=color, fontsize=10,
422            fontweight='bold')
423
424 plt.savefig(os.path.join(figures_dir, 'f12_q1_vs_q2_cost.png'), dpi
425             =300, bbox_inches='tight')
426 plt.close()
427
428 # F13
429 metrics = ['Total km']
430 q1_m = [sum_q1['total_km']]
431 q2_m = [sum_q2['total_km']]
432
433 fig, ax = plt.subplots(figsize=(5.5, 5))
434 x = np.arange(len(metrics))
435 bars1 = ax.bar(x - width / 2, q1_m, width, label='Q1',
436               color=SCENARIO_COLORS['Q1'], edgecolor='white',
437               linewidth=0.8)
438 bars2 = ax.bar(x + width / 2, q2_m, width, label='Q2',

```

```

436         color=SCENARIO_COLORS['Q2'], edgecolor='white',
linewidth=0.8)
437     ax.set_xticks(x)
438     ax.set_xticklabels(metrics)
439     ax.set_ylabel('Value')
440     ax.legend(loc='upper right')
441     ax.grid(axis='x', visible=False)
442
443     y_max = max(max(q1_m), max(q2_m))
444     y_min = 9000
445     ax.set_ylim(y_min, y_max * 1.08)
446     pad = (ax.get_ylim()[1] - ax.get_ylim()[0]) * 0.03
447     for b, v in zip(bars1, q1_m):
448         ax.text(b.get_x() + b.get_width() / 2, b.get_height() + pad,
449                 f'{v:,.1f}', ha='center', va='bottom', fontsize=10,
450                 color=SCENARIO_COLORS['Q1'], fontweight='bold')
451     for b, v in zip(bars2, q2_m):
452         ax.text(b.get_x() + b.get_width() / 2, b.get_height() + pad,
453                 f'{v:,.1f}', ha='center', va='bottom', fontsize=10,
454                 color=SCENARIO_COLORS['Q2'], fontweight='bold')
455
456     plt.savefig(os.path.join(figures_dir, 'f13_q1_vs_q2_emissions.png'),
457                 dpi=300, bbox_inches='tight')
458     plt.close()
459
460 # =====
461 # F16: E-Fleet 灵敏度
462 # =====
463 def plot_f16(base_dir):
464     efleet_df = pd.read_csv(os.path.join(base_dir, 'results', 'sensitivity'
465     , 'e_fleet.csv'))
466
467     fig, ax = plt.subplots(figsize=(8.5, 6))
468     line_color = HUE_PAIRS['red'][0]
469     muted = HUE_PAIRS['red'][1]
470
471     ax.plot(efleet_df['E-3000'], efleet_df['total_cost'],
472             color=line_color, linewidth=2.0, zorder=2)
473
474     min_idx = int(efleet_df['total_cost'].idxmin())
475     sizes = [110 if i == min_idx else 60 for i in range(len(efleet_df))]
476     face_colors = [line_color if i == min_idx else 'white' for i in range(
477     len(efleet_df))]
478     edge_colors = [line_color] * len(efleet_df)
479     ax.scatter(efleet_df['E-3000'], efleet_df['total_cost'],

```

```

477         s=sizes, facecolor=face_colors, edgecolor=edge_colors,
478         linewidth=2.0, zorder=3)
479
480     ax.fill_between(efleet_df['E-3000'], efleet_df['total_cost'],
481                    efleet_df['total_cost'].max(),
482                    color=muted, alpha=0.35, zorder=1)
483
484     min_x = efleet_df['E-3000'].iloc[min_idx]
485     min_y = efleet_df['total_cost'].iloc[min_idx]
486     ax.annotate(f'min: ¥{min_y:,.0f}\n@ E-3000 = {min_x}',
487               xy=(min_x, min_y),
488               xytext=(15, 25), textcoords='offset points',
489               fontsize=10, color=line_color, fontweight='bold',
490               arrowprops=dict(arrowstyle='->', color=line_color, lw=1.0))
491
492     ax.set_xlabel('E-3000 Fleet Size')
493     ax.set_ylabel('Total Cost (¥)')
494     plt.savefig(os.path.join(base_dir, 'figures', 'f16_sensitivity_efleet.
495                             png'),
496               dpi=300, bbox_inches='tight')
497
498     plt.close()
499
500 # =====
501 # F17: 灵敏度曲线
502 # =====
503 def plot_f17(base_dir):
504     fuel_df = pd.read_csv(os.path.join(base_dir, 'results', 'sensitivity',
505                                         'fuel_price.csv'))
506     carbon_df = pd.read_csv(os.path.join(base_dir, 'results', 'sensitivity',
507                                           'carbon_price.csv'))
508
509     base_fuel = fuel_df[fuel_df['mult'] == 1.0]['total_cost'].values[0]
510     base_carbon = carbon_df[carbon_df['mult'] == 1.0]['total_cost'].values
511     [0]
512
513     # Convert each sweep into (parameter deviation %, cost change %) —
514     # full curves, not just endpoints
515     fuel_dev = (fuel_df['mult'].values - 1.0) * 100
516     fuel_pct = (fuel_df['total_cost'].values - base_fuel) / base_fuel * 100
517     carbon_dev = (carbon_df['mult'].values - 1.0) * 100
518     carbon_pct = (carbon_df['total_cost'].values - base_carbon) /
519     base_carbon * 100
520
521     fig, ax = plt.subplots(figsize=(9.5, 6))

```

```

516 fuel_color = HUE_PAIRS['orange'][0]
517 fuel_muted = HUE_PAIRS['orange'][1]
518 carbon_color = HUE_PAIRS['purple'][0]
519 carbon_muted = HUE_PAIRS['purple'][1]
520
521 # 燃油曲线
522 ax.plot(fuel_dev, fuel_pct, color=fuel_color, linewidth=2.2, zorder=3,
label='Fuel Price')
523 ax.scatter(fuel_dev, fuel_pct, s=70, facecolor='white', edgecolor=
fuel_color,
524             linewidth=2.0, zorder=4)
525 ax.fill_between(fuel_dev, fuel_pct, 0, color=fuel_muted, alpha=0.55,
zorder=1)
526
527 # 碳曲线
528 ax.plot(carbon_dev, carbon_pct, color=carbon_color, linewidth=2.2,
zorder=3, label='Carbon Price')
529 ax.scatter(carbon_dev, carbon_pct, s=70, facecolor='white', edgecolor=
carbon_color,
530             linewidth=2.0, zorder=4)
531 ax.fill_between(carbon_dev, carbon_pct, 0, color=carbon_muted, alpha
=0.55, zorder=1)
532
533 # 通过基准线的参考线 (0%偏差, 0%成本变化)
534 ax.axhline(0, color='#7F8C8D', linewidth=0.9, linestyle='--', zorder=2)
535 ax.axvline(0, color='#7F8C8D', linewidth=0.9, linestyle='--', zorder=2)
536
537 def _annotate_endpoints(devs, pcts, color):
538     for d, p in [(devs[0], pcts[0]), (devs[-1], pcts[-1])]:
539         offset = (10, 8) if p >= 0 else (10, -14)
540         ax.annotate(f'{p:+.2f}%', xy=(d, p), xytext=offset,
541                     textcoords='offset points', fontsize=9.5,
542                     color=color, fontweight='bold')
543
544 _annotate_endpoints(fuel_dev, fuel_pct, fuel_color)
545 _annotate_endpoints(carbon_dev, carbon_pct, carbon_color)
546
547
548 ax.scatter([0], [0], s=110, facecolor=HUE_PAIRS['navy'][0], edgecolor='
white', linewidth=1.8, zorder=5, label='Baseline')
549 ax.set_xlabel('Parameter Deviation from Baseline (%)')
550 ax.set_ylabel('Change in Total Cost (%)')
551 ax.legend(loc='best')
552 ax.grid(axis='x', alpha=0.4)
553

```

```

554     plt.savefig(os.path.join(base_dir, 'figures', 'f17_sensitivity_tornado.
555     png'),
556                 dpi=300, bbox_inches='tight')
557     plt.close()
558     # =====
559     # F18: 成本布局 — 碳价
560     # =====
561     def plot_f18(base_dir):
562         carbon_df = pd.read_csv(os.path.join(base_dir, 'results', 'sensitivity'
563         , 'carbon_price.csv'))
564
565         fig, ax = plt.subplots(figsize=(8.5, 6))
566         line_color = HUE_PAIRS['blue'][0]
567         muted = HUE_PAIRS['blue'][1]
568
569         ax.fill_between(carbon_df['mult'], carbon_df['total_cost'],
570                        carbon_df['total_cost'].min(),
571                        color=muted, alpha=0.5, zorder=1)
572         ax.plot(carbon_df['mult'], carbon_df['total_cost'],
573                color=line_color, linewidth=2.0, zorder=2)
574
575         base_row = carbon_df[carbon_df['mult'] == 1.0]
576         is_base = (carbon_df['mult'] == 1.0).values
577         sizes = np.where(is_base, 130, 60)
578         face_colors = [HUE_PAIRS['orange'][0] if b else 'white' for b in
579         is_base]
580         edge_colors = [HUE_PAIRS['orange'][0] if b else line_color for b in
581         is_base]
582         ax.scatter(carbon_df['mult'], carbon_df['total_cost'],
583                   s=sizes, facecolor=face_colors, edgecolor=edge_colors,
584                   linewidth=2.0, zorder=4)
585
586         if not base_row.empty:
587             bx = base_row['mult'].values[0]
588             by = base_row['total_cost'].values[0]
589             ax.annotate(f'Baseline: ¥{by:,.0f}',
590                        xy=(bx, by), xytext=(12, -22), textcoords='offset
591             points',
592                        fontsize=10, color=HUE_PAIRS['orange'][0], fontweight='
593             bold',
594                        arrowprops=dict(arrowstyle='->', color=HUE_PAIRS['
595             orange'][0], lw=1.0))
596
597         legend_handles = [

```

```

592     plt.Line2D([0], [0], marker='o', color='w', markerfacecolor='white'
,
593                 markededgecolor=line_color, markersize=8,
markededgewidth=2,
594                 label='Carbon Price Varied'),
595     plt.Line2D([0], [0], marker='o', color='w', markerfacecolor=
HUE_PAIRS['orange'][0],
596                 markededgecolor=HUE_PAIRS['orange'][0], markersize=10,
597                 label='Baseline (mult = 1.0)'),
598 ]
599 ax.legend(handles=legend_handles, loc='best')
600
601 ax.set_xlabel('Carbon Price Multiplier')
602 ax.set_ylabel('Total Cost (¥)')
603
604 plt.savefig(os.path.join(base_dir, 'figures', 'f18_pareto_cost_vs_co2.
png'),
605             dpi=300, bbox_inches='tight')
606 plt.close()
607
608 # =====
609 # F19 & F20: 蒙特卡洛模拟结果
610 # =====
611 def plot_f19_f20(base_dir):
612     mc_dir = base_dir / 'results' / 'montecarlo'
613     df = pd.read_csv(mc_dir / 'q2_mc.csv')
614     with open(mc_dir / 'summary.json', 'r') as f:
615         summary = json.load(f)
616
617     M = len(df)
618
619     cost_mean = df['cost'].mean()
620     cost_std = df['cost'].std()
621     p05 = df['cost'].quantile(0.05)
622     p95 = df['cost'].quantile(0.95)
623
624     fig, ax = plt.subplots(figsize=(9, 5.2))
625     counts, bin_edges, patches = ax.hist(df['cost'], bins=15,
626                                           color=HUE_PAIRS['orange'][1],
627                                           edgecolor='white', linewidth=0.8)
628     for i, p in enumerate(patches):
629         if bin_edges[i] <= cost_mean < bin_edges[i + 1]:
630             p.set_facecolor(HUE_PAIRS['orange'][0])
631
632     ax.axvline(cost_mean, color=HUE_PAIRS['red'][0], linestyle='--',

```

```

linewidth=2.0,
633         label=f"Mean: ¥{cost_mean:,.0f}")
634     ax.axvline(p05, color='#7F8C8D', linestyle=':', linewidth=1.6,
635         label=f"5th pct: ¥{p05:,.0f}")
636     ax.axvline(p95, color='#7F8C8D', linestyle=':', linewidth=1.6,
637         label=f"95th pct: ¥{p95:,.0f}")
638
639     ax.text(0.98, 0.97, f"N = {M}\n = ¥{cost_std:,.0f}",
640         transform=ax.transAxes, ha='right', va='top', fontsize=10,
641         bbox=dict(boxstyle='round,pad=0.4', facecolor='white',
642             edgecolor='#D5DBDB', alpha=0.95))
643
644     ax.set_xlabel('Total Cost (¥)')
645     ax.set_ylabel('Frequency')
646     ax.legend(loc='upper left')
647     plt.savefig(os.path.join(base_dir, 'figures', 'f19_mc_cost_distribution
648 .png'),
649         dpi=300, bbox_inches='tight')
650     plt.close()
651
652     total_packets = 148
653     mean_late = summary['mean_late']
654     on_time = max(total_packets - mean_late, 0.0)
655
656     fig, ax = plt.subplots(figsize=(9, 3.4))
657     ax.barh([0], [on_time], color=HUE_PAIRS['green'][1], edgecolor='white',
658         linewidth=0.8,
659         label=f'On-time: {on_time:.1f}')
660
661     ax.barh([0], [mean_late], left=[on_time], color=HUE_PAIRS['red'][0],
662         edgecolor='white', linewidth=0.8, label=f'Late: {mean_late:.1f}
663 ')
664
665     ax.text(on_time / 2, 0, f'{on_time:.1f}', ha='center', va='center',
666         color=HUE_PAIRS['green'][0], fontsize=11, fontweight='bold')
667     if mean_late > 0:
668         ax.text(on_time + mean_late / 2, 0, f'{mean_late:.1f}', ha='center'
669 , va='center',
670         color='white', fontsize=11, fontweight='bold')
671
672     ax.set_xlim(0, total_packets)
673     ax.set_ylim(-0.6, 0.6)
674     ax.set_yticks([])
675     ax.set_xlabel(f'Packets (out of {total_packets})')
676     ax.legend(loc='upper right', bbox_to_anchor=(1.0, 1.40), ncol=2,
677         frameon=False)

```

```

672     ax.grid(axis='y', visible=False)
673     ax.spines['left'].set_visible(False)
674
675     plt.savefig(os.path.join(base_dir, 'figures', 'f20_mc_late_probability.
png'),
676                 dpi=300, bbox_inches='tight')
677     plt.close()
678
679 if __name__ == "__main__":
680     print("Regenerating all 14 figures from saved data...")
681     _apply_style()
682     figures_dir = base_dir / 'figures'
683     figures_dir.mkdir(parents=True, exist_ok=True)
684
685     # 加载所需数据
686     data_dir = base_dir / 'data'
687     coords, D, df_orders, tw, customers = ingest_data(data_dir)
688
689     precomp_dir = base_dir / 'results' / 'precomputed'
690     G_ij = np.load(precomp_dir / 'G_ij.npy')
691     with open(precomp_dir / 'packets.json', 'r') as f:
692         packets = json.load(f)
693
694     with open(base_dir / 'results' / 'q1' / 'summary.json', 'r') as f:
695         sum_q1 = json.load(f)
696
697     with open(base_dir / 'results' / 'q2' / 'summary.json', 'r') as f:
698         sum_q2 = json.load(f)
699
700     with open(base_dir / 'results' / 'phaseB_solution_q2.json', 'r') as f:
701         routes_q2 = json.load(f)
702
703     # 执行绘图
704     plot_f01(customers, figures_dir)
705     plot_f02(figures_dir)
706     plot_f03(G_ij, figures_dir)
707     plot_f04(customers, packets, figures_dir)
708     plot_f08(sum_q1, figures_dir)
709     plot_f10(sum_q1, figures_dir)
710     plot_f11(routes_q2, customers, figures_dir)
711     plot_f12_f13(sum_q1, sum_q2, figures_dir)
712     plot_f16(base_dir)
713     plot_f17(base_dir)
714     plot_f18(base_dir)
715     plot_f19_f20(base_dir)

```

```
716  
717     print(f"All 14 figures successfully regenerated in: {figures_dir}")
```